Indian Institute of Technology Jodhpur

Summer Research Internship 2026

Research Report

Surveillance Drone Path Planning



Prepared by:

Aman Shukla

(Summer Research Intern)

B.Tech Electronics Engineering

Bharat Ratan Sardar Vallabhbhai Patel

Rajkiya Engineering College, Basti

Under the Guidance of:

Prof. Binod Kumar

Department of Electronics Engineering

Indian Institute of Technology, Jodhpur

Duration:

June 2026 – July 2026

Table of Contents

Chapter 1. Introduction Page No.

1.1. Autonomous Drone Navigation……...07

1.2. Need of Path Planning…….07

1.3. Role of Computer Vision…….07

1.4. Raspberry Pi Camera Rev 1.3……..08-09

1.5. Image Processing…….09

1.6. Obstacle Grid……...09

1.7. A* Algorithm……...10

1.8. Planned Path……10

Chapter 2. Objectives……**10-11**

Chapter 3. Hardware and Software Setup

3.1. Hardware Components……....11-13

3.2.Software Environment…….....13-15

Chapter 4. Raspberry Pi OS Installation and Configuration

4.1. Raspberry Pi Imager Setup…….15-24

4.2.Initial Boot and Network Setup……...24-25

Chapter 5. Remote Access Setup

5.1. SSH Configuration…….....25-26

5.2.IP Address Detection……26-27

5.3. VNC Viewer Setup……..27-28

Chapter 6. Raspberry Pi Interface Configuration

6.1. Enable Camera Interface..…….. .28-30

6.2. Enable VNC Interface……...……...30-31

6.3. Apply Configuration Settings……..31

Chapter 7. Pi Camera Setup and Testing

7.1.Camera Hardware Connection..…… . .31-32
7.2.Camera Detection……..…….....32-33
7.3. Image Capture……..33-34

Chapter 8. OpenCV Installation and Image Processing

8.1. OpenCV Verification……....35
8.2.Reading Image Using Python…….……...…….....35-36
8.3. Grayscale Conversion……..37-38
8.4. Thresholding……....38-39

Chapter 9. Camera Image to Grid Conversion……...39-41

Chapter 10. A* Path Planning Algorithm……..41-42

Chapter 11. Grid Size Experiments

11.1 10×10 Grid ……43-44
11.2 20×20 Grid……44-46
11.3 50×50 Grid……46-47
11.4 Performance Comparison……...48
11.5 Execution Time Graph……...48-49

Chapter 12. Grid Size Experiments

12.1. Camera Based Path Planning Approach……49-50
12.2. Camera Based A* Result……...50-51

Chapter 13. Results and Observation

13.1. What Worked…….52
13.2. Effect of Grid Size (10×10, 20×20, 50×50)……. 52
13.3.Camera Integration Result…….53

Chapter 14. Challenges Faced During Work

14.1. Camera Not Detected…….....53
14.2.SD Card Reader Issue…….....53
14.3. SSH and VNC Connection Issues…….....54

14.4. Package Installation Issue (OpenCV)…….. . . . 54

Chapter 15. Future Work…….....54-55

Chapter 16. Conclusion…….....55

Chapter 17. Appendices

17.1 A-Important Commands Used…….....56-57

17.2 B-Important Code and Files…….....57-60

List of Figures

Figure No.	Figure Title	Page No.
Figure 1.1	Overall System Architecture	08
Figure 1.2	Raspberry Pi Camera Rev 1.3 Module	09
Figure 1.3	Raspberry Pi 4	12
Figure1.4	External SD Card Slot	13
Figure 1.5	Raspberry Pi Hardware Setup	13
Figure 1.6	Raspberry Pi OS	14
Figure 1.7	Thonny IDE	14
Figure 1.8	VNC Viewer	15
Figure 1.9	External SD card reader	16
Figure 1.10	Raspberry Pi Imager Interface	16
Figure 1.11	Raspberry Pi OS Interface	17
Figure 1.12	Storage Selection in Raspberry Pi Imager	17
Figure 1.13	Hostname Configuration	18
Figure 1.14	Localization Configuration	19
Figure 1.15	User Account Configuration	19
Figure 1.16	Wi-Fi Configuration	20
Figure 1.17	SSH Remote Access Configuration	21
Figure 1.18	Raspberry Pi Connect Setup	21
Figure 1.19	Raspberry Pi Connect Sign-in Window	22
Figure 1.20	Write OS Image Option	22
Figure 1.21	Erase and Write Confirmation	23
Figure 1.22	OS Installation and Verification Progress	23
Figure 1.23	SD Card Inserted into Raspberry Pi	24
Figure 1.24	Raspberry Pi First Boot	24

Figure No.	Figure Title	Page No.
Figure 1.25	Raspberry Pi and PC Connected to Same Hotspot Network	25
Figure 1.26	SSH Connection Through Terminal	26
Figure 1.27	Raspberry Pi IP Address Detection	27
Figure 1.28	RealVNC Viewer Connection	27
Figure 1.29	Raspberry Pi Desktop Through VNC	28
Figure 1.30	Raspberry Pi Configuration Menu (raspi-config)	29
Figure 1.31	VNC Interface Enable Option	30
Figure 1.32	VNC Interface Enable Option	31
Figure 1.33	Raspberry Pi Camera Rev 1.3 Connected to CSI Port	32
Figure 1.34	Camera Detection Output (rpicam-hello Command)	33
Figure 1.35	Image Capture and Verification Using rpicam-still	34
Figure 1.36	OpenCV Verification	35
Figure 1.37	Image Dimension Verification Using OpenCV	36
Figure 1.38	Grayscale Image Output	38
Figure 1.39	Threshold Image Output	39
Figure 1.40	Generated Grid Output (Obstacle Map)	41
Figure 1.41	A* Algorithm Flowchart	42
Figure 1.42	10×10 Grid Map Generated for A* Path Planning	44
Figure 1.43	20×20 Grid Map with Obstacles	46
Figure 1.44	50×50 Grid Map with Obstacles	47
Figure 1.45	Grid Size vs Execution Time	49
Figure 1.46	Camera Based A Processing Flowchart	50
Figure 1.47	Camera Based A Result	51

Chapter 1. Introduction

Autonomous navigation is a key aspect of today's robots, including drones and mobile robots. A drone must understand what is around it, steer clear of obstacles, and choose a safe route to get to its destination. This project centers on mapping obstacles with a camera and planning paths using Raspberry Pi, OpenCV, and the A* algorithm.

1.1 Autonomous Drone Navigation

From my understanding, autonomous drone navigation means enabling a drone to move on its own from a starting position to a destination without continuous human input. The drone takes input from sensors or a camera and processes that information to make movement decisions.

In this work, I used a camera to capture images of the surrounding area. These images are then processed to identify obstacles, and this information is used further for generating a safe path for navigation.

1.2 Need of Path Planning

Path planning is required so that the drone can reach its destination safely without hitting obstacles. When the environment is unknown, the system must calculate a route that avoids obstacles and also tries to find the shortest or most efficient path.

For this, I used the A* algorithm, which works by calculating the best possible path based on the actual distance covered and the estimated distance to the goal. This helps in finding an optimal and safe route.

1.3 Role of Computer Vision

Computer vision is used to help the system “see” and understand the environment using images captured by the camera. It processes the image to extract useful information like obstacles and free space.

In my implementation, OpenCV was used to process the captured images. The image is converted into grayscale and then into a binary format, which is then converted into a grid representation. This grid is finally used by the A* algorithm for path planning.

In this work, **OpenCV** is used for image processing operations such as image conversion and obstacle detection. The processed output is converted into a grid map, which is further used by the A* algorithm for path planning.

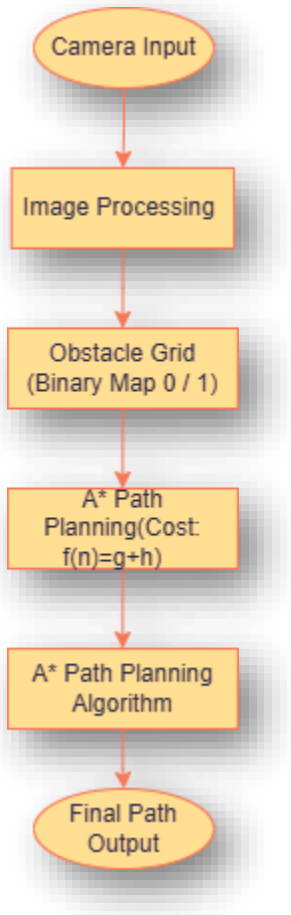


Figure: 1.1. System Architecture of Camera-Based A Path Planning

Overall System Architecture

The system works in a simple step-by-step flow. First, the camera captures the environment image. Then I process that image using OpenCV to extract useful information. After that, the processed image is converted into a grid where obstacles and free space are marked. This grid is then given to the A* algorithm to calculate the best possible path. Finally, the system generates and shows the final safe path from start to goal.

1.4 Raspberry Pi Camera Rev 1.3

The **Raspberry Pi Camera Rev 1.3** is a small camera module that directly connects to the Raspberry Pi through the CSI port. I used it mainly to capture real-time images of the surrounding environment for my project.

In my implementation, the camera acts as the starting point of the system. The images captured from it are processed using OpenCV to detect obstacles and separate free space. After processing, the output is converted into a grid format, which is then used by the A* algorithm for path planning.



Figure:1.2. Raspberry Pi Camera Rev 1.3 Module

1.5 Image Processing

After capturing the image using the camera, I process it using OpenCV to extract useful information from the scene. The main idea is to simplify the image so that obstacles and free space can be clearly separated.

In my work, I apply basic steps like converting the image into grayscale and then applying thresholding. This helps in clearly distinguishing the obstacle areas from the free path, which is later used for grid formation and path planning.

1.6 Obstacle Grid

After image processing, I convert the output into a simple grid format for easier path planning. The whole environment is divided into small cells, where each cell represents a part of the scene.

In this grid, I mark free space and obstacles separately so that the A* algorithm can easily understand where it can move and where it should avoid. This grid becomes the main input for path planning in my project.

1.7 A* Algorithm

In my project, I use the A* algorithm on the generated obstacle grid to find the best possible path from the start point to the goal. It works by checking different possible routes and selecting the one with the lowest cost while avoiding obstacles.

The algorithm considers both the actual distance traveled and the estimated distance to the goal, which helps it decide an efficient and safe path.

1.8 Planned Path

The final output of my system is the path generated by the A* algorithm. This path shows the safest and most efficient route from the start point to the destination while avoiding all detected obstacles.

This planned path represents how the system would guide a drone or robot to move step by step in a real environment.

Chapter 2. Objectives

The main goal of this project is to build a camera-based system that can map obstacles and plan a safe path using Raspberry Pi, OpenCV, and the A* algorithm. The focus is to understand how real-world images can be converted into a simple grid and then used for path planning.

The objectives of this work are:

- To set up Raspberry Pi with camera support for image-based navigation applications.
- To capture images of the environment using Raspberry Pi Camera.
- To process captured images using OpenCV for identifying obstacles and free space.
- To convert the processed image into a grid-based obstacle map.

- To implement the A* algorithm for finding an optimal path between source and destination points.
- To perform experiments with different grid sizes and observe the effect on path planning.
- To study the integration of computer vision and path planning techniques for autonomous navigation systems.

Chapter 3. Hardware and Software Setup

In this chapter, I have described all the hardware parts and software tools that I used to build and test the camera-based obstacle mapping and A* path planning system. This includes the Raspberry Pi setup, camera module, and the required software environment for image processing and algorithm implementation.

3.1 Hardware Components

For my project, I used a simple hardware setup that includes Raspberry Pi, camera module, SD card, and a laptop. Each component plays an important role in running and testing the system.

Raspberry Pi acts as the main processing unit, the camera is used to capture images of the environment, the SD card stores the operating system and programs, and the laptop is used for coding, remote access, and monitoring the output.

Raspberry Pi

Raspberry Pi is the main processing unit of my system. I used it to run the operating system and handle all the processing tasks of the project.

In my implementation, it is responsible for capturing images from the camera, running OpenCV operations, and executing the A* path planning algorithm to generate the final path.

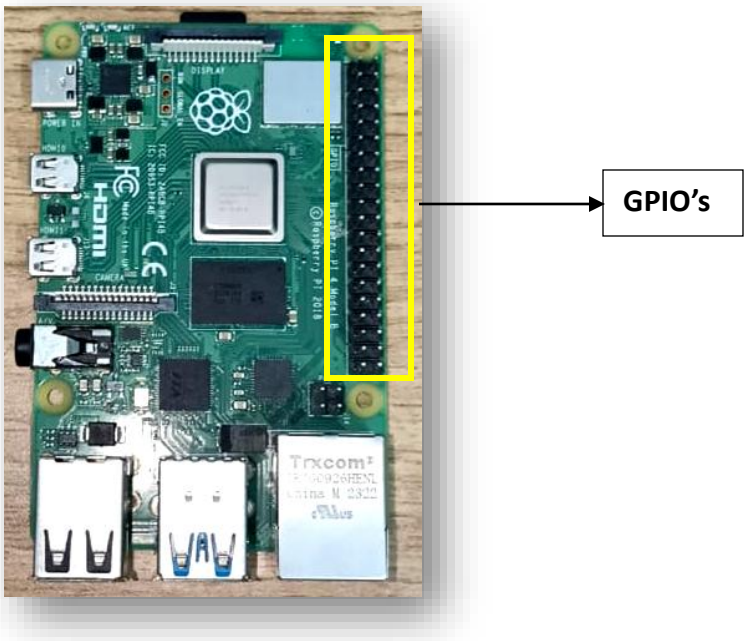


Figure1.3: Raspberry Pi 4

Raspberry Pi Camera V2

The Raspberry Pi Camera V2 is used in my project to capture images of the surrounding environment. These captured images act as the main input for the system.

I use these images for further processing in OpenCV, where obstacles and free space are identified, and then the processed data is used for generating the grid and path planning using the A* algorithm.

SD Card

An SD card is used for installing **Raspberry Pi OS** and storing required software, libraries, programs, and project files.

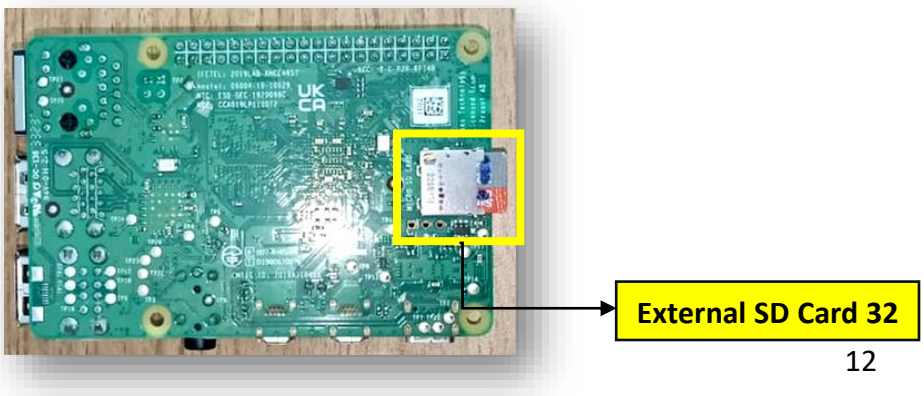


Figure 1.4: External SD Card Slot

Laptop

In my project, the laptop is used as the main control and development device. I used it to configure the Raspberry Pi, write and run code, and monitor the output during testing.

It is also used for remote access through SSH and VNC, which allows me to control the Raspberry Pi system without connecting a separate monitor or keyboard.

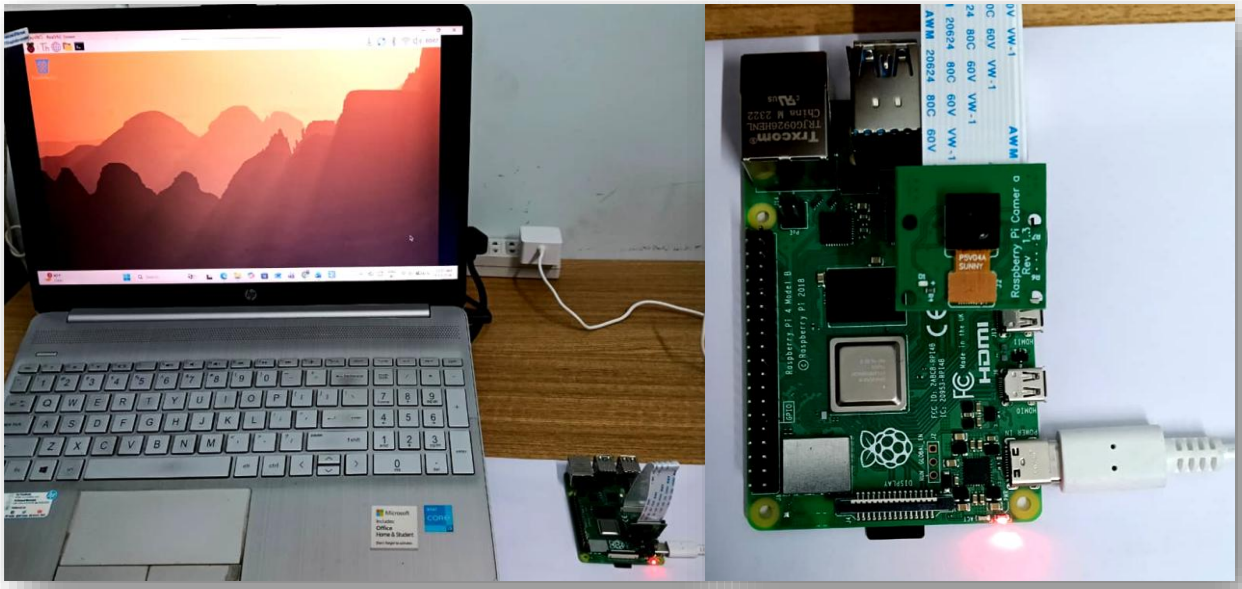


Figure 1.5: Raspberry Pi Hardware Setup

3.2 Software Environment

The software environment refers to all the software tools, operating system, and applications used in this project to develop, run, and test the system, including programming tools and remote access applications required for implementation.

Raspberry Pi OS

Raspberry Pi OS is installed on the SD card and provides the working environment for running Python programs, OpenCV libraries, and other required packages.

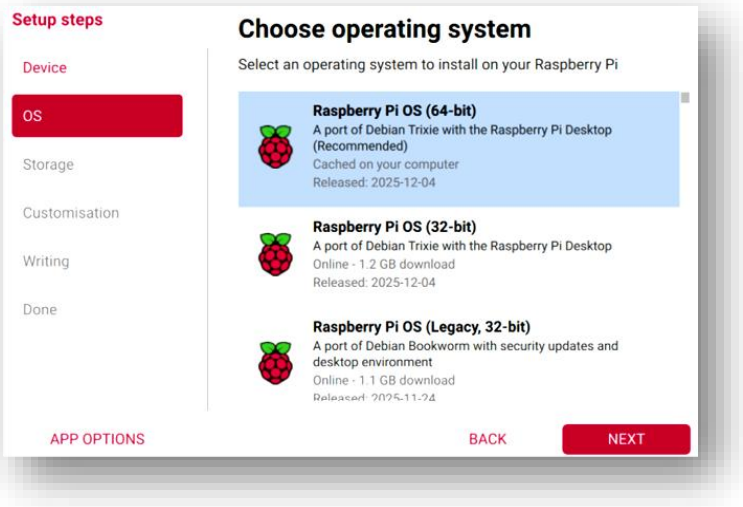


Figure 1.6: Raspberry Pi OS

Thonny IDE

Thonny is used for **writing and executing** Python programs on Raspberry Pi. It helps in testing image processing and A* algorithm code.

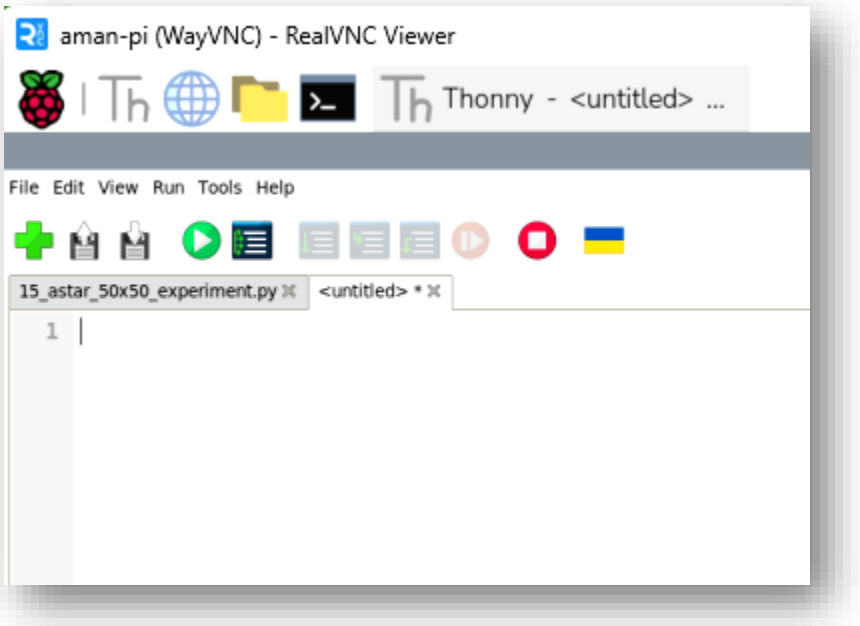


Figure1.7: Thonny IDE

VNC Viewer

VNC Viewer is used for remote desktop access of Raspberry Pi. It allows controlling the **Raspberry Pi interface** from a laptop without connecting a separate display.

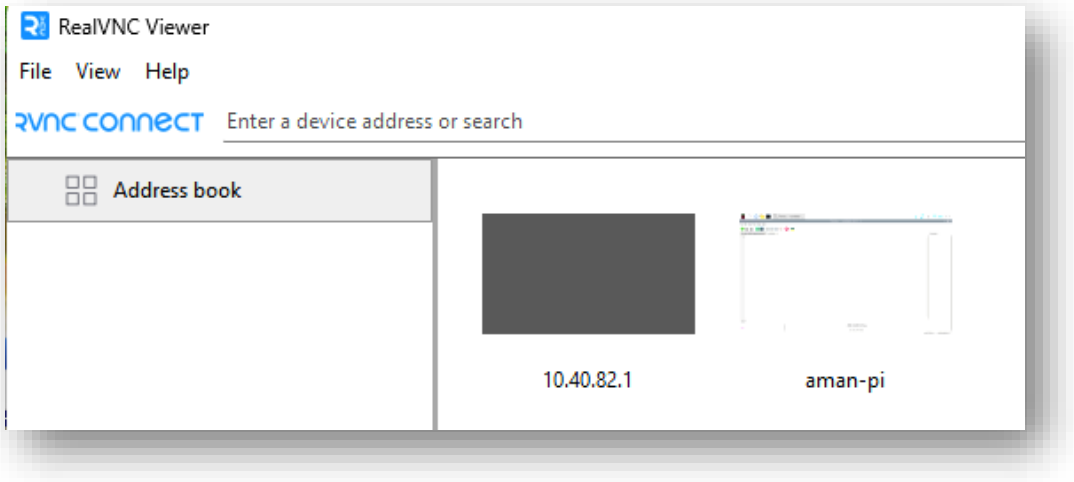


Figure 1.8: VNC Viewer

Chapter 4. Raspberry Pi OS Installation and Configuration

Raspberry Pi OS was installed on the SD card using Raspberry Pi Imager. The SD card was connected to the laptop using an external SD card reader. The installed OS was then used to boot and configure the Raspberry Pi.

4.1 Raspberry Pi Imager Setup

Before installing Raspberry Pi OS, an **SD card** was inserted into an external SD card reader and connected to the laptop.

The Raspberry Pi **Imager software** was downloaded and installed from the official Raspberry Pi website. After successful installation, the application was opened to begin the OS installation process.



Figure 1.9: External SD card reader

Step 1: Open Raspberry Pi Imager

After opening Raspberry Pi Imager, the main window appears. The required Raspberry Pi device model was selected from the device selection option **as shown in Figure 1.10.**

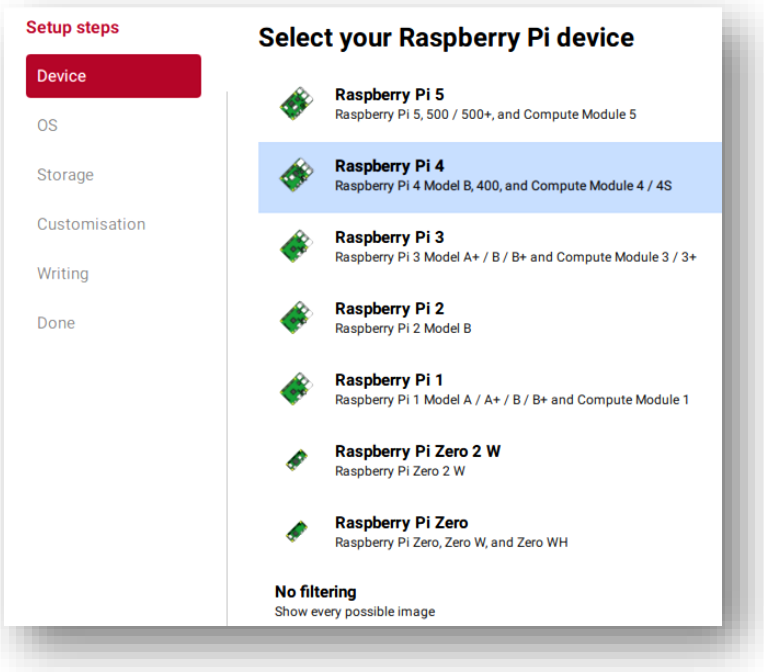


Figure 1.10: Raspberry Pi Imager Interface

Step 2: Select Raspberry Pi Device

The required Raspberry Pi model was selected from the device selection option.

Step 3: Select Operating System

Raspberry Pi OS was selected as the operating system for the Raspberry Pi.

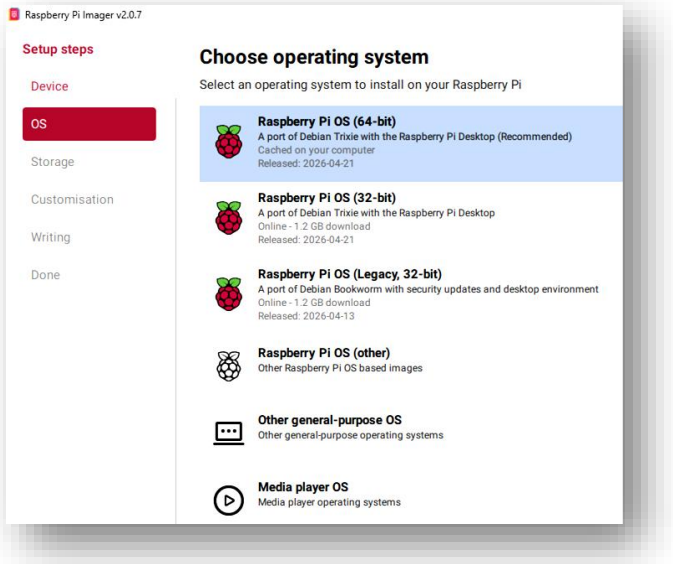


Figure 1.11: Raspberry Pi OS Interface

Step 4: Select Storage

After selecting the Raspberry Pi device and operating system, the connected SD card was selected as the storage device **as shown in Figure 1.12.**

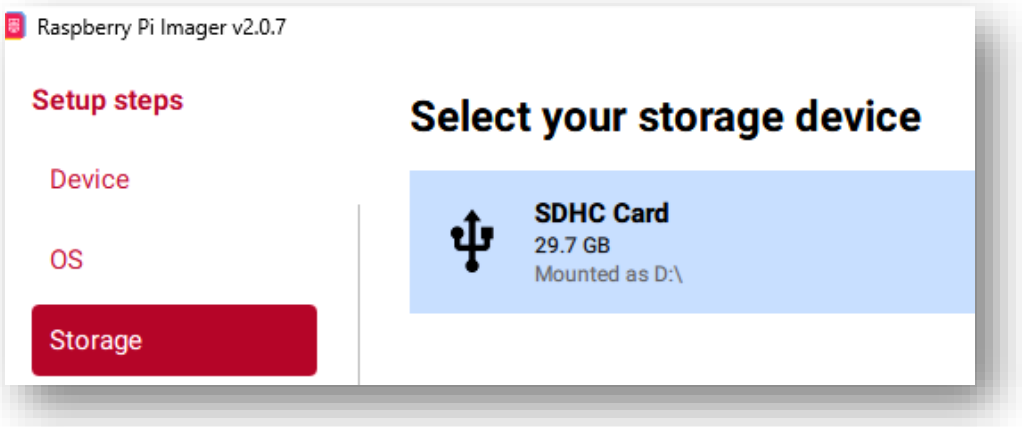


Figure 1.12: Storage Selection in Raspberry Pi Imager

Step 2: Operating System Customization

After selecting the **storage device**, the customization window was opened to configure the Raspberry Pi settings before writing the OS image.

In **customization settings**, the hostname of Raspberry Pi was configured. The hostname configuration window is shown in **Figure 1.13.**

Example: Hostname: **aman-pi**

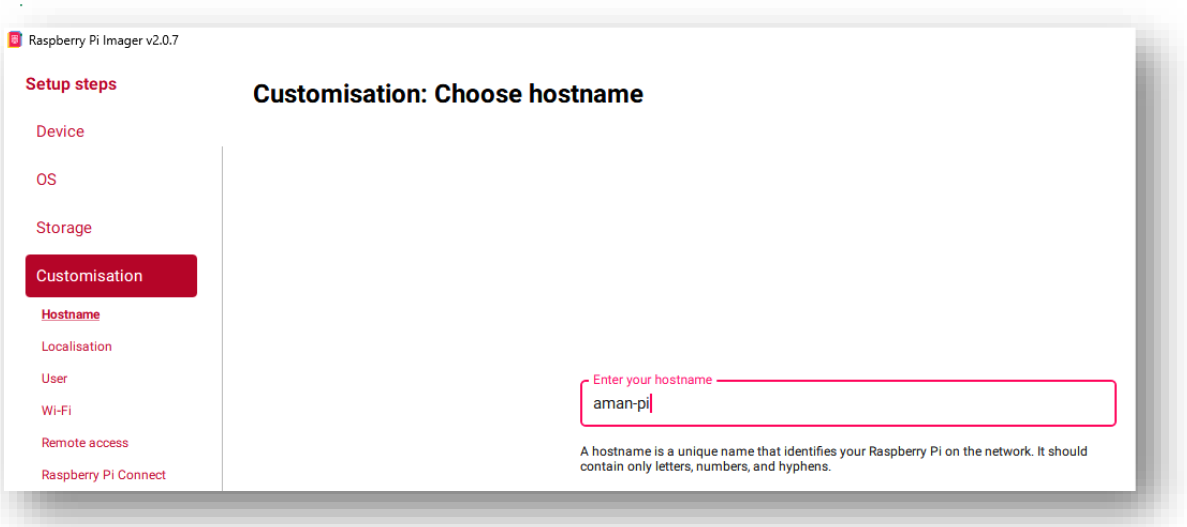


Figure 1.13: Hostname Configuration

Step 3: Localization Settings

After setting the hostname, localization settings were configured. The location was selected to automatically set the time zone and keyboard layout.

Selected settings:

- Capital city: New Delhi, India
- Time zone: Asia/Kolkata
- Keyboard layout: IN

The localization window is shown in **Figure 1.14.**

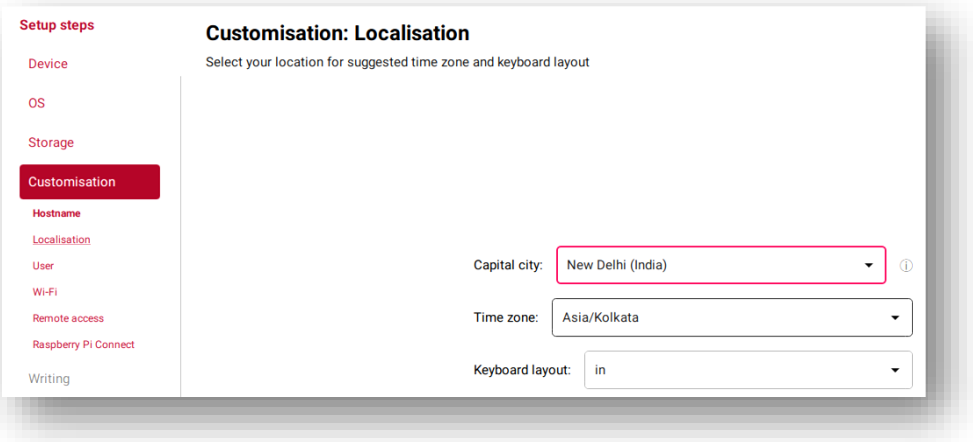


Figure 1.14: Localization Configuration

Step 4: User Account Configuration

After localization settings, a user account was created for Raspberry Pi access.

Example:

Username: aman

Password: xyz@111

Confirm Password: xyz@111

The password was confirmed and the next option was selected **as shown in Figure 1.15.**

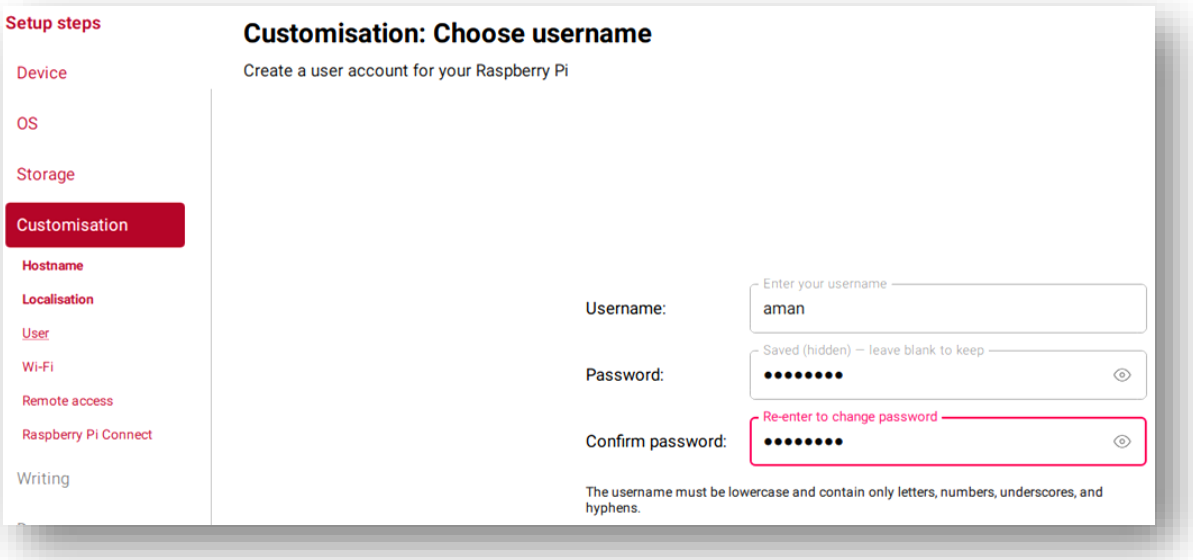


Figure 1.15: User Account Configuration

Step 5: Wi-Fi Configuration

The Wi-Fi network details were entered to allow Raspberry Pi to connect with the network during first boot.

Network details:

SSID: Hotspot/Network Name

Password: xyz@1234

Confirm Password: xyz@1234

After entering the details, the next option was selected. The Wi-Fi configuration window is shown in **Figure 1.16.**

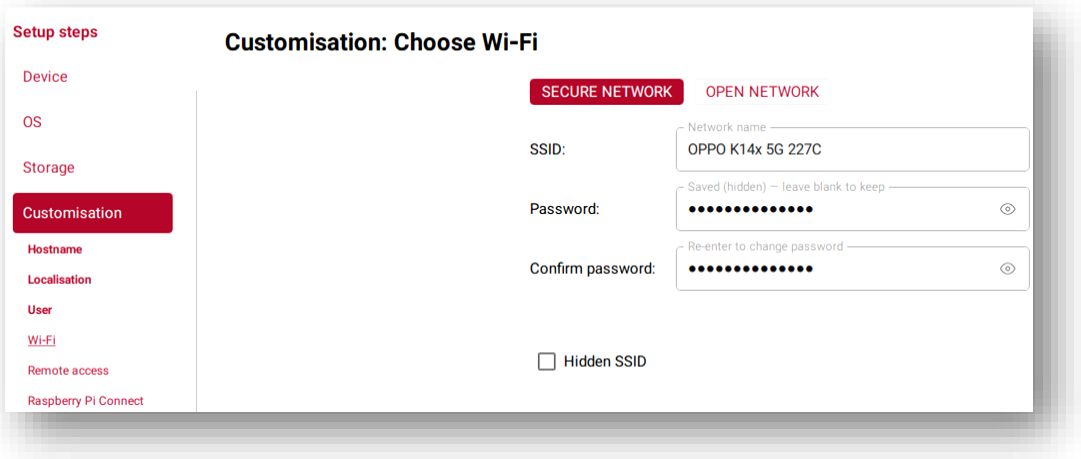


Figure 1.16: Wi-Fi Configuration

Step 6: Remote Access Configuration (SSH)

SSH was enabled for remote access of Raspberry Pi.

The option Enable SSH was selected.

Two authentication methods were available:

- Password authentication
- Public key authentication

Password authentication was selected for this setup. The SSH configuration window is shown in **Figure 1.17.**

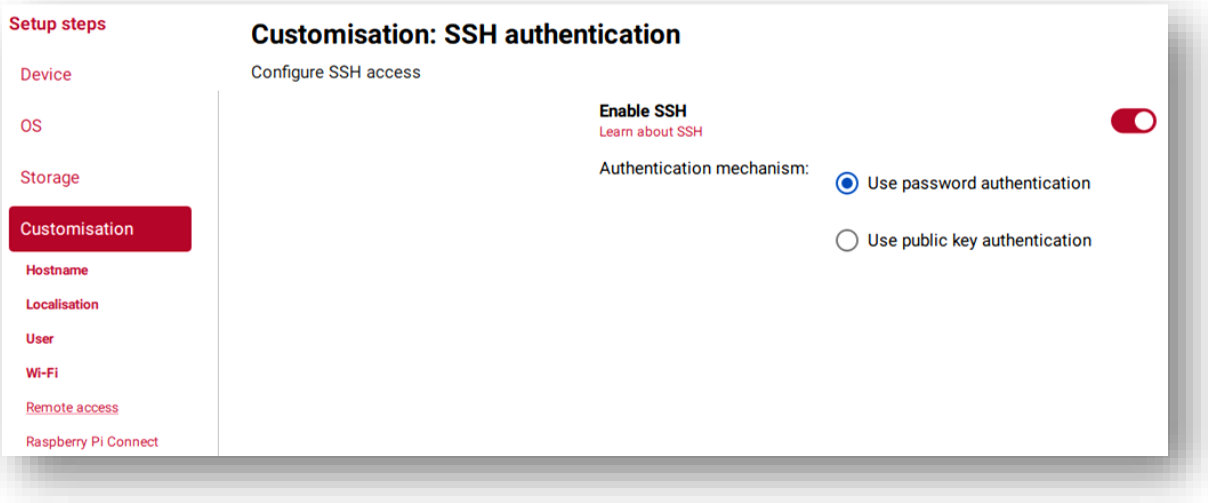


Figure 1.17: SSH Remote Access Configuration

Step 7: Raspberry Pi Connect Setup

Raspberry Pi Connect was enabled for remote access. The account was created using an email ID and signed in to generate the required token.

After selecting Raspberry Pi Connect, the sign-in window appeared. The Raspberry Pi Connect setup window is shown in **Figure 1.18.**

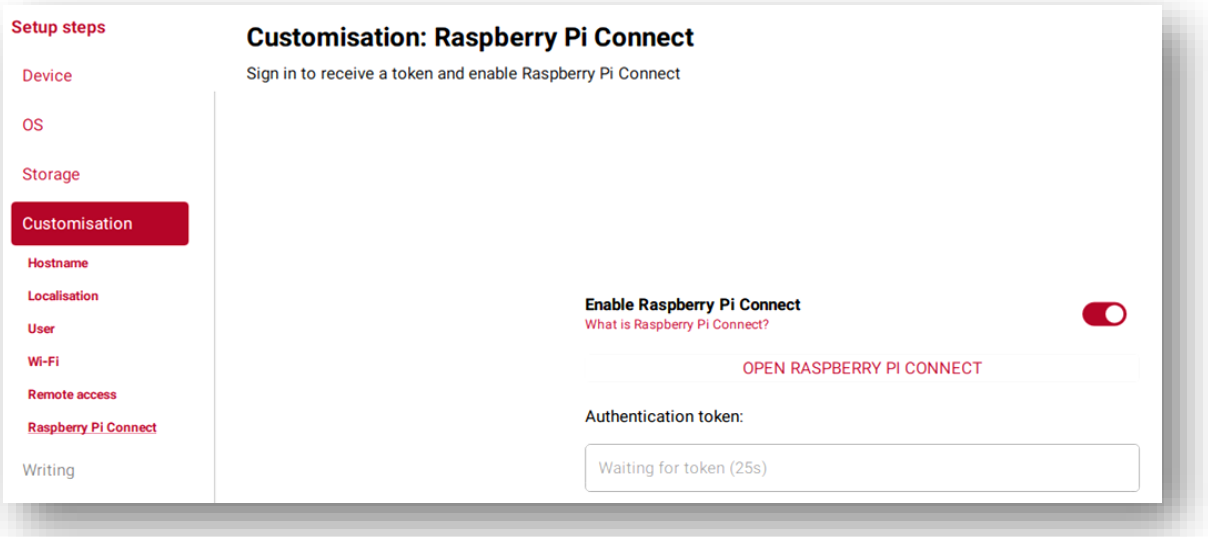


Figure 1.18: Raspberry Pi Connect Setup

After clicking Raspberry Pi Connect, the sign-in page appeared as shown in **Figure 1.19.**

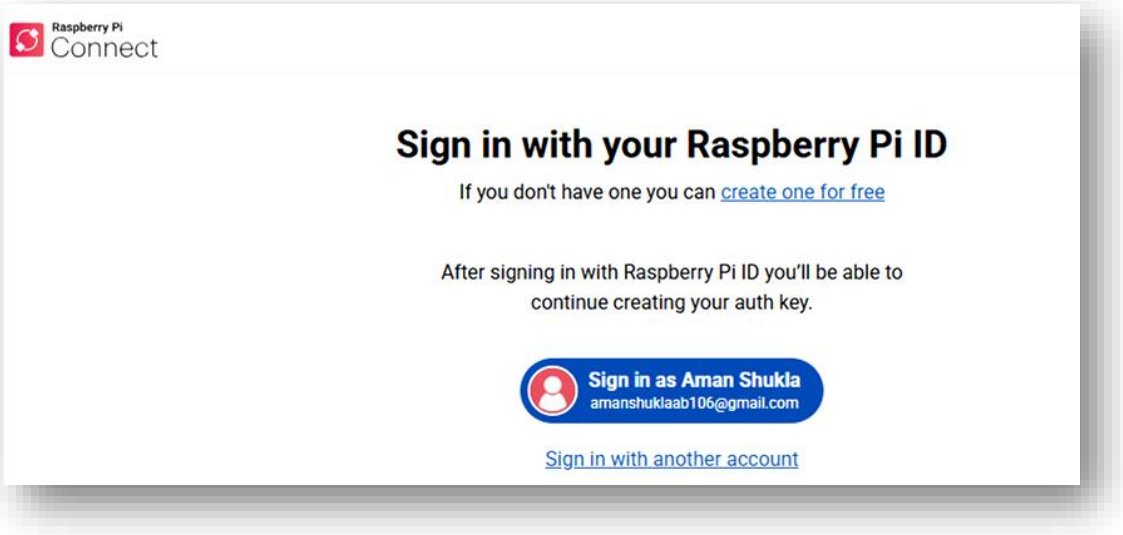


Figure 1.19: Raspberry Pi Connect Sign-in Window

Step 9: Writing Raspberry Pi OS Image

After completing all customization settings, the final configuration summary was displayed.

The **Write option** was selected to start installing Raspberry Pi OS on the SD card.

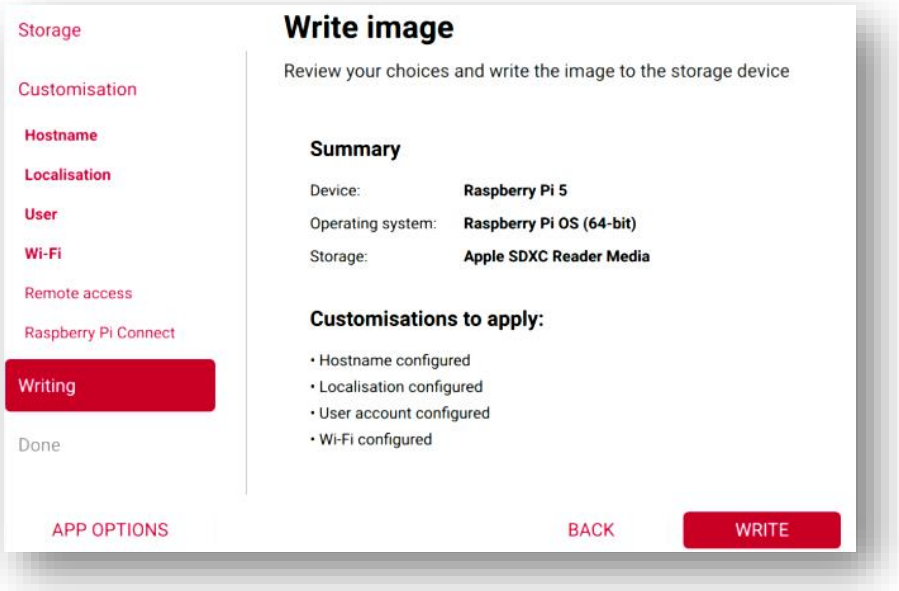


Figure 1.20: Write OS Image Option

A confirmation window appeared showing that all data on the SD card would be erased.

The option "I understand, erase and write" was selected to continue.

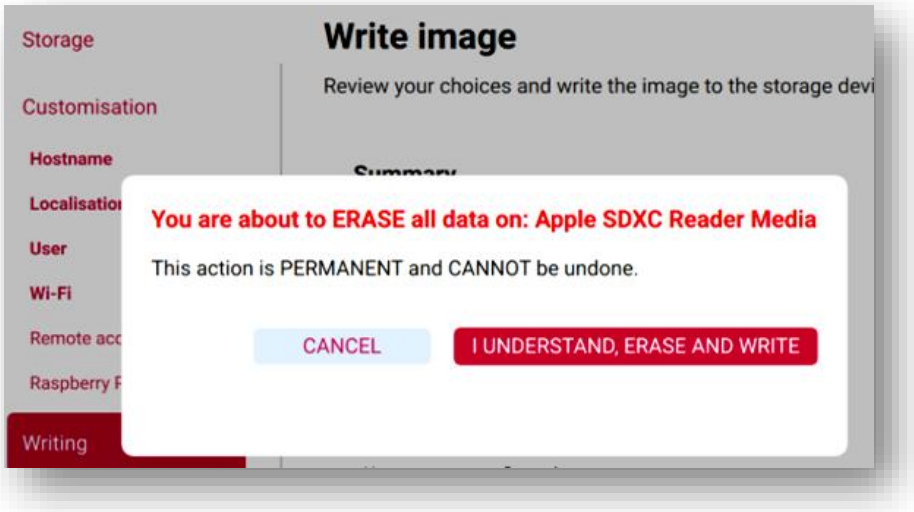


Figure 1.21: Erase and Write Confirmation

The Raspberry Pi Imager started writing the OS image to the SD card, The installation progress was displayed, followed by verification of the written image.

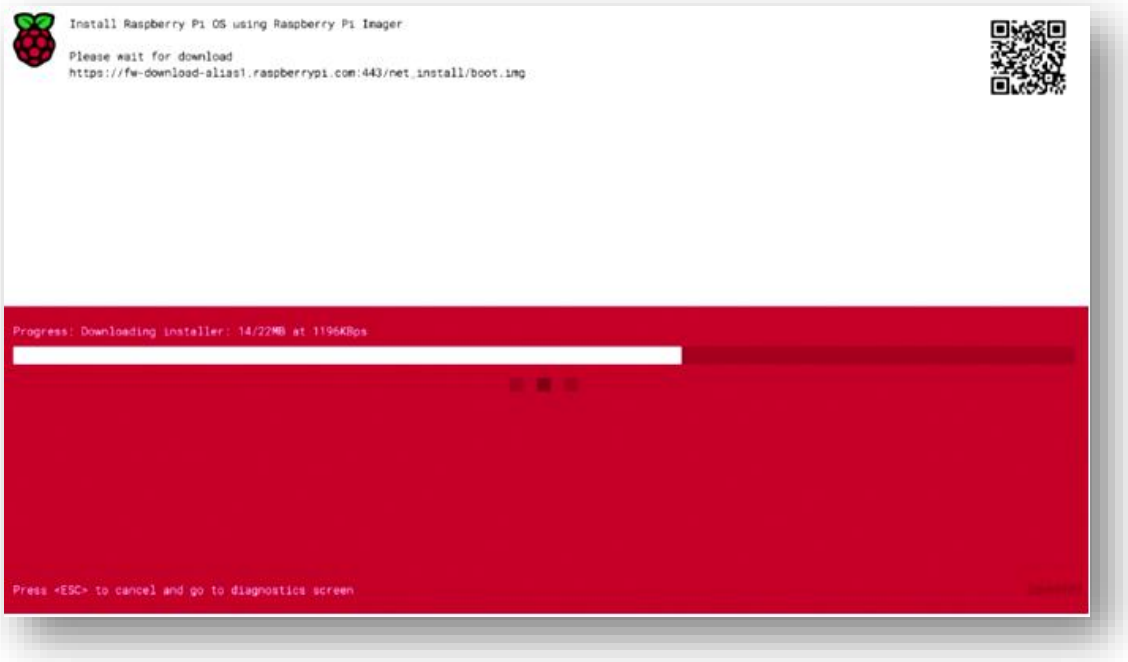


Figure 1.22: OS Installation and Verification Progress

Step 9: Insert SD Card into Raspberry Pi

After successful installation and verification, the SD card was safely removed from the card reader and inserted into the SD card slot on the Raspberry Pi board.



Figure 1.23: SD Card Inserted into Raspberry Pi

4.2 Initial Boot and Network Setup

After completing the Raspberry Pi OS installation, I inserted the SD card into the Raspberry Pi and powered it on for the first boot.

During the first boot, the system started with the installed OS and automatically used the Wi-Fi settings that I had configured earlier to connect to the network.

For remote access, I connected both the Raspberry Pi and my laptop to the same mobile hotspot. This allowed both devices to communicate properly and made it possible for me to access the Raspberry Pi using SSH and VNC.

The Raspberry Pi first boot screen is shown **as shown in Figure 1.24.**

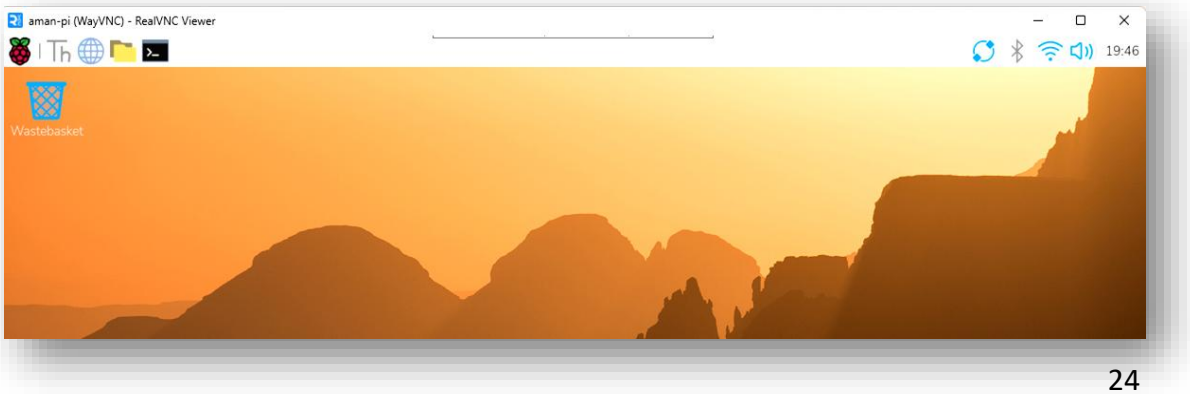


Figure 1.24: Raspberry Pi First Boot

The network connection setup with the same hotspot is shown **as shown in Figure 1.25.**

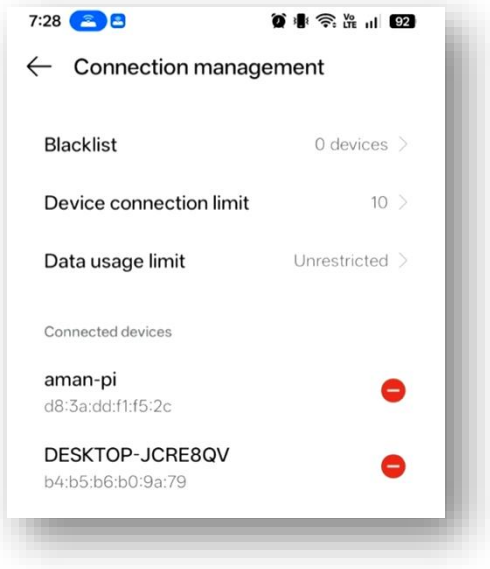


Figure 1.25: Raspberry Pi and Laptop Connected to Same Hotspot Network

Chapter 5. Remote Access Setup

Remote access was set up so that I could control the Raspberry Pi directly from my laptop without using a separate monitor, keyboard, or mouse.

For this purpose, I used SSH and VNC, which allowed me to access both the terminal and the graphical interface of the Raspberry Pi remotely and manage the system easily.

5.1 SSH Configuration

SSH (Secure Shell) is a network protocol that allows secure access to another computer through a command-line interface. It is mainly used to run commands and control the Raspberry Pi remotely over the same network.

In my project, I enabled SSH during the Raspberry Pi OS setup. Both the Raspberry Pi and my laptop were connected to the same mobile hotspot so they could communicate with each other.

After that, I established the SSH connection by opening the terminal on my laptop and entering the required command.

Command:ssh aman@aman-pi.local

After entering the command, I provided the Raspberry Pi username and password to successfully complete the connection.

The SSH connection established through the terminal is shown in **Figure 1.26.**

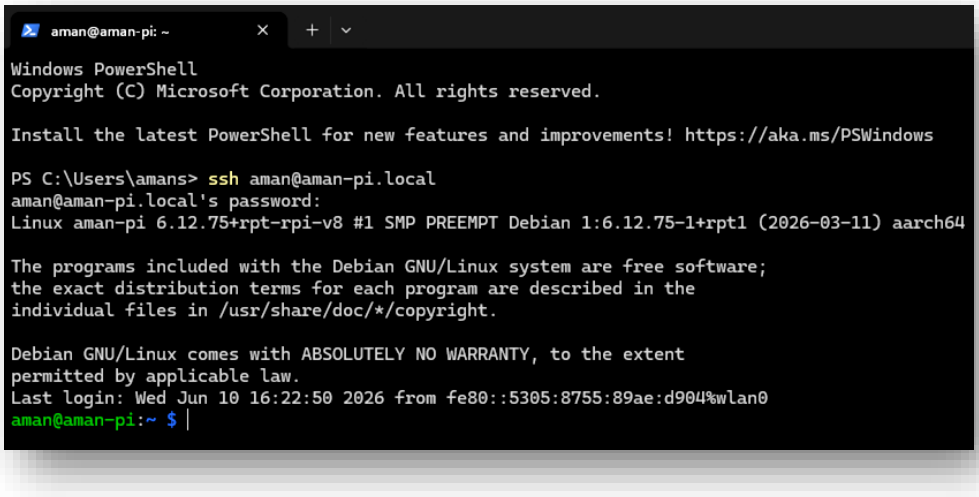


Figure 1.26: SSH Connection Through Terminal

5.2 IP Address Detection

The IP address of Raspberry Pi was identified to establish communication between Raspberry Pi and laptop on the same network. The IP address is used for remote access when required.

The following commands were used in Raspberry Pi terminal to check device information:

Command: hostname

The hostname command displays the configured name of the Raspberry Pi.

The **hostname -I** command displays the IP address assigned to the Raspberry Pi on the connected network.

The **Raspberry Pi IP address** obtained through the terminal is shown **as shown in Figure 1.27.**

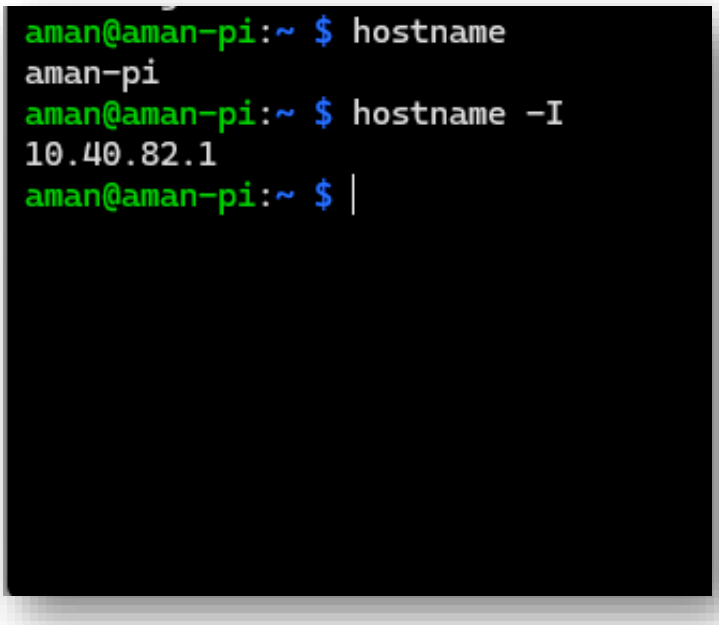


Figure 1.27: Raspberry Pi IP Address Detection

5.3 VNC Viewer Setup

VNC (Virtual Network Computing) is a remote desktop access technology used to view and control the graphical interface of another computer over a network connection.

In this project, VNC Viewer was used to access the Raspberry Pi desktop from the laptop. Both Raspberry Pi and laptop were connected to the same mobile hotspot network.

The Raspberry Pi hostname/IP address was entered in VNC Viewer to establish the remote desktop connection.

The VNC Viewer connection window is shown as shown in **Figure 1.28**

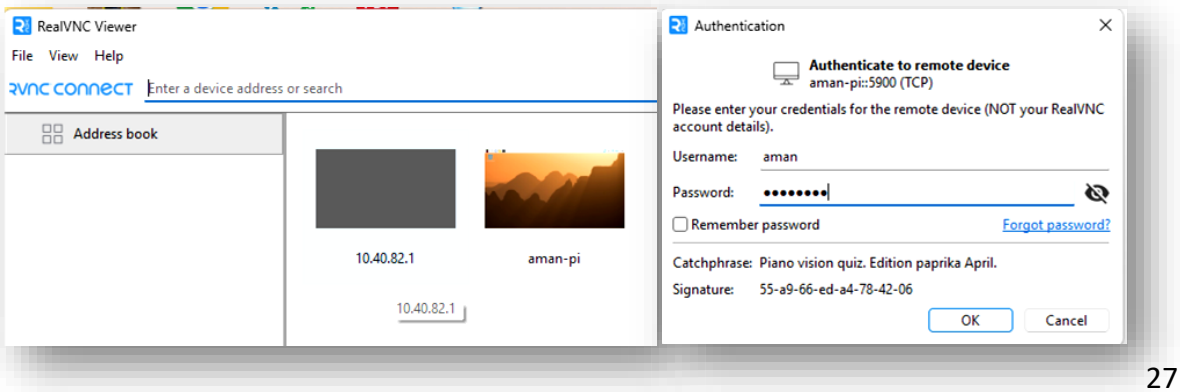


Figure 1.28. RealVNC Viewer Connection

After successful connection, the Raspberry Pi desktop was displayed on the laptop screen. The Raspberry Pi graphical interface accessed through VNC is shown as shown in **Figure 1.29**.

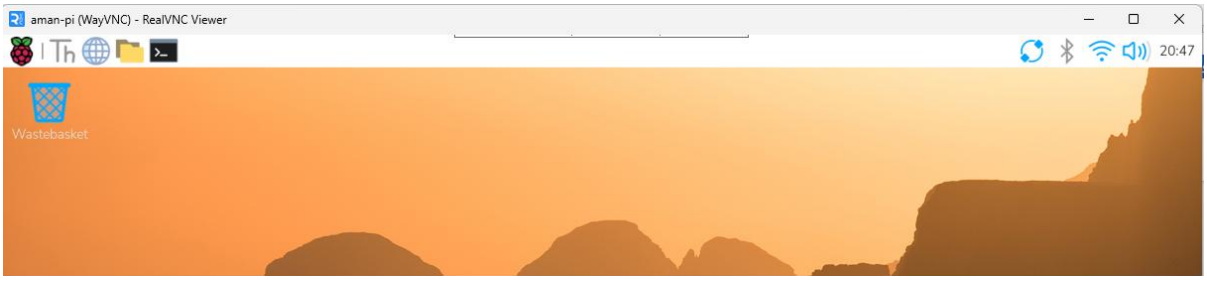


Figure 1.29: Raspberry Pi Desktop Through VNC

Chapter 6. Raspberry Pi Interface Configuration

The Raspberry Pi configuration tool was used to enable required hardware and remote access settings. It allows changing system settings such as camera support, interfaces, and other Raspberry Pi options.

The configuration menu was opened using the **command:**

Step 1. Open VNC Viewer and paste the command in the raspberry pi terminal

Command: **sudoraspi-config**

Password: **xyz@111**

After executing the command, the Raspberry Pi configuration window appeared.

The raspi-config menu is shown as shown in **Figure 1.30.**

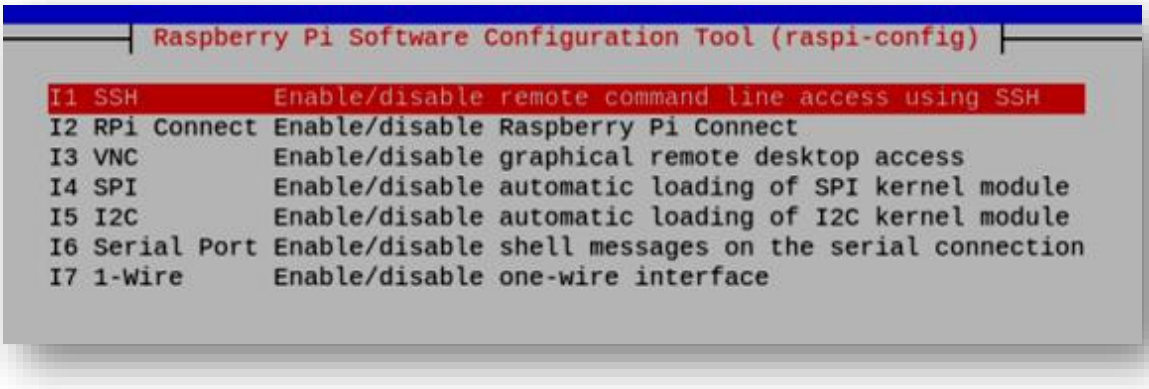


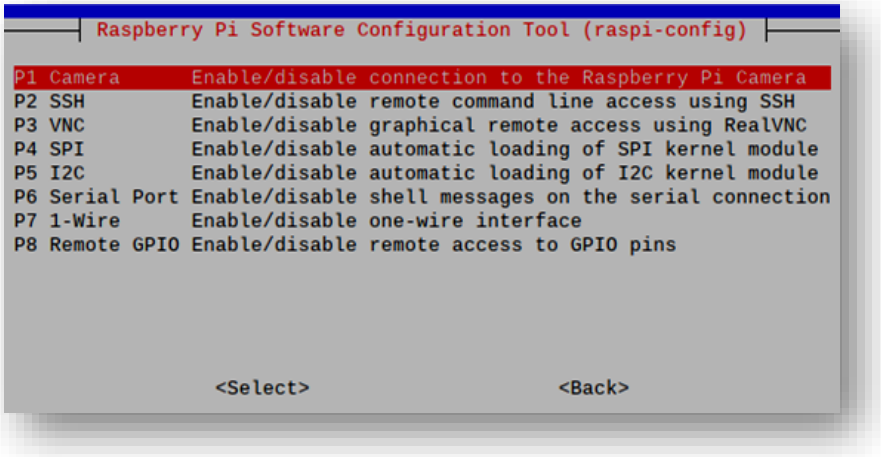
Figure 1.30: Raspberry Pi Configuration Menu (raspi-config)

6.1 Enable Camera Interface

To use the Raspberry Pi Camera module, the camera interface was enabled from the configuration menu.

From Interface Options, the Camera option was selected and enabled. This allowed the Raspberry Pi to access the connected camera module for image capturing.

The camera enable option is shown in **Figure 1.31.**



Click on the P1 camera to enable the raspberry pi camera, after clicking on the P1 Camera the next window will show the camera interface is enable, click on ok

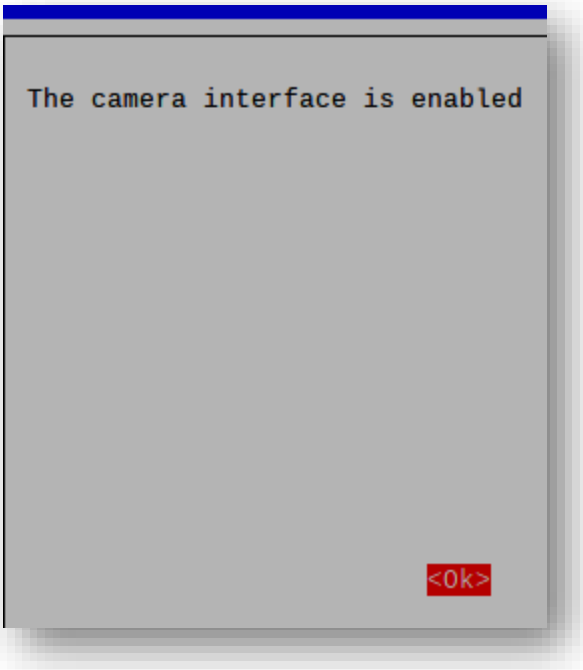


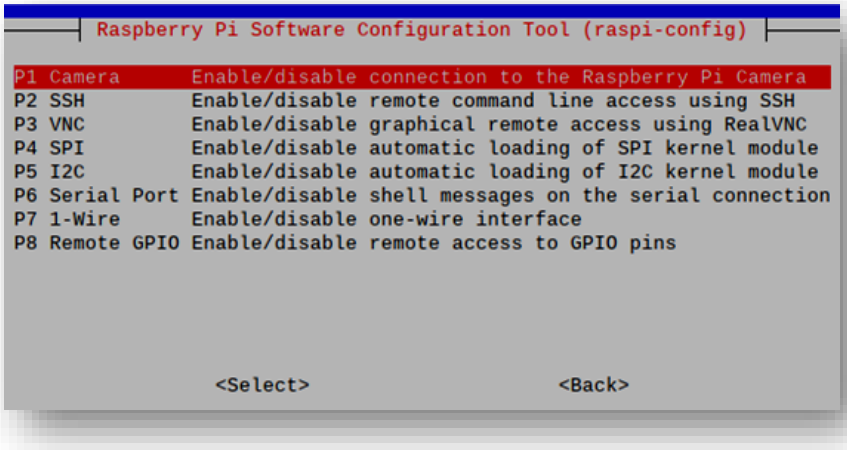
Figure 1.31: VNC Interface Enable Option

6.2 Enable VNC Interface

For remote desktop access, the VNC interface was enabled from the configuration menu.

From Interface Options, the VNC option was selected and enabled. This allowed the Raspberry Pi desktop to be accessed remotely using VNC Viewer.

The VNC enable option is shown in **Figure 1.32.**



After opening the configure window click on the P3 VNC – to enable the remote desktop access

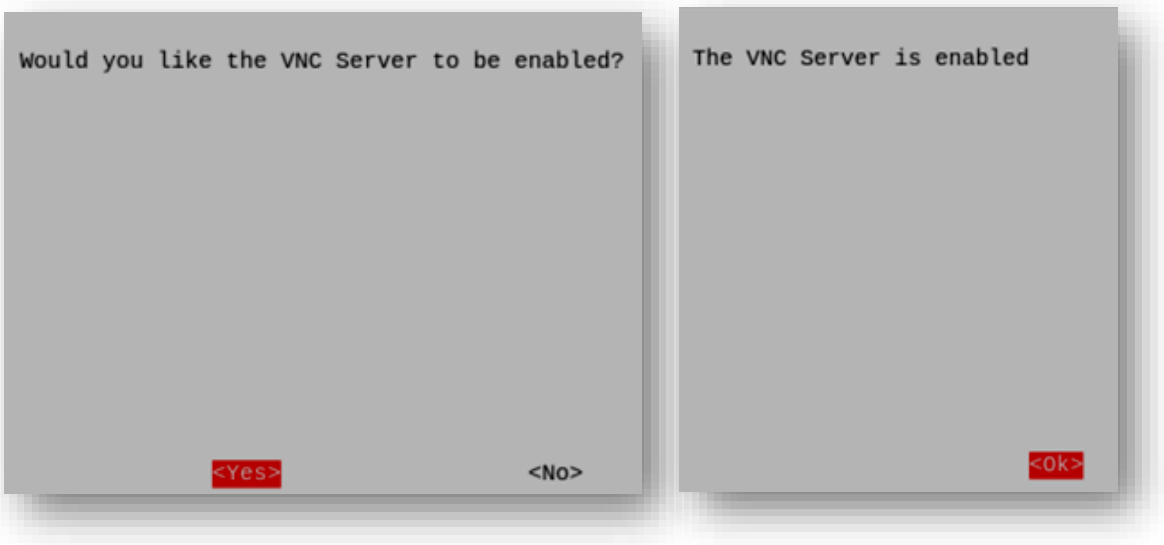


Figure 1.32: VNC Interface Enable Option

After clicking on the P3 VNC the next popup window will ask you to for YES/NO, click on the YES, and the next pop-up window will show VNC Server is enabled

6.3 Apply Configuration Settings

After enabling the required interfaces, the configuration settings were saved and the Raspberry Pi was restarted to apply the changes.

Chapter 7. Pi Camera Setup and Testing

The Raspberry Pi Camera Rev 1.3 was connected and tested to capture images for image processing and obstacle detection.

7.1 Camera Hardware Connection

The **Raspberry Pi Camera Rev 1.3** was connected to the Raspberry Pi through the Camera Serial Interface (CSI) port.

To insert the camera ribbon cable into the CSI port, the silver side of the ribbon cable was kept towards the **HDMI side** of the Raspberry Pi. The ribbon cable was inserted in the correct orientation and the connector lock was properly closed to secure the connection.

The connected camera module with Raspberry Pi is shown as shown in **Figure 1.33.**



Figure 1.33: Raspberry Pi Camera Rev 1.3 Connected to CSI Port

7.2 Camera Detection

After connecting the Raspberry Pi Camera Rev 1.3, the camera detection test was performed using the following command:

The following command was used in the Raspberry Pi terminal:

Command: rpicam-hello

The command successfully detected the connected camera module. The terminal output showed the camera sensor information as **OV5647**, confirming that the Raspberry Pi Camera Rev 1.3 was properly connected and recognized by the Raspberry Pi.

The camera detection output is shown **as shown in Figure 1.34.**

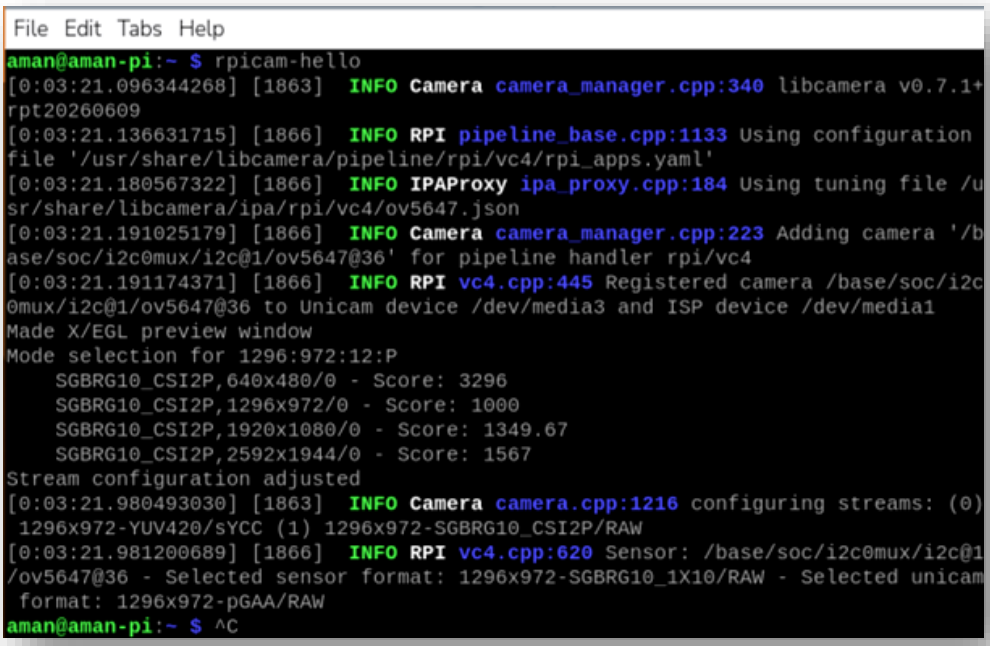


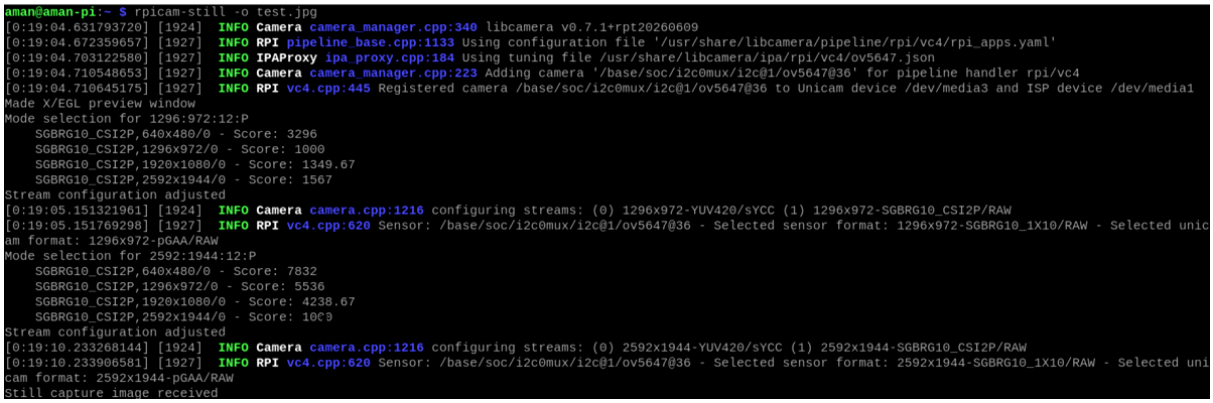
Figure 1.34: Camera Detection Output (rpicam-hello Command)

7.3 Image Capture

After successful camera detection, an image capture test was performed using the Raspberry Pi Camera.

The following command was used in the Raspberry Pi terminal:

Command: rpicam-still -o test.jpg



The command captures an image and saves it with the name **test.jpg** in the current working directory, To check the location of the captured image, the following command was used:

Command: pwd

```
aman@aman-pi:~ $ pwd
/home/aman
aman@aman-pi:~ $ ls
astar_path_result.jpg  Documents  gray.jpg  Pictures  Templates  threshold.jpg
Desktop                Downloads  Music     Public    test.jpg   Videos
aman@aman-pi:~ $ xdg-open test.jpg

(eom:2119): EOM-WARNING **: 19:09:25.178: Error loading Peas typelib: Typelib file for namespace 'Peas', version '1.0' not found

(eom:2119): EOM-WARNING **: 19:09:25.178: Error loading PeasGtk typelib: Typelib file for namespace 'PeasGtk', version '1.0' not found
```

Image 2: Checking saved location/file

This command displays the current directory path where the image is saved. To verify that the captured image file is present, the following command was used:

Command: ls

The output shows the saved image file **test.jpg**. The captured image was opened using the command:

Command: xdg-open test.jpg

The image captured from the Raspberry Pi Camera is displayed and verified successfully, and image capture process and saved image verification are shown in **Figure 1.35**.

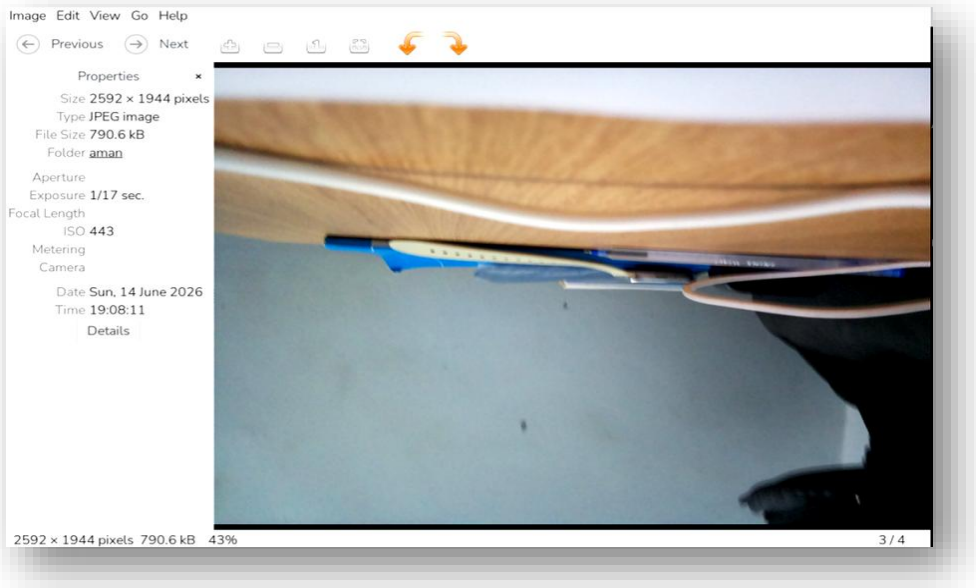


Figure 1.35: Image Capture and Verification Using rpicam-still

Chapter 8. OpenCV Installation and Image Processing

OpenCV (Open Source Computer Vision Library) was used for image processing in this project. It provides functions to read, process, and analyze images captured using the Raspberry Pi Camera.

8.1 OpenCV Verification

After installing OpenCV, I verified the installation by importing the OpenCV library in Python.

I opened the Python interpreter using the command:

Command:python

After entering the Python environment, the **OpenCV library** was imported using:

Command:import cv2

The command executed successfully without any error, confirming that OpenCV was installed properly on Raspberry Pi and ready for image processing applications.

The OpenCV verification process is shown **as shown in Figure 1.36.**

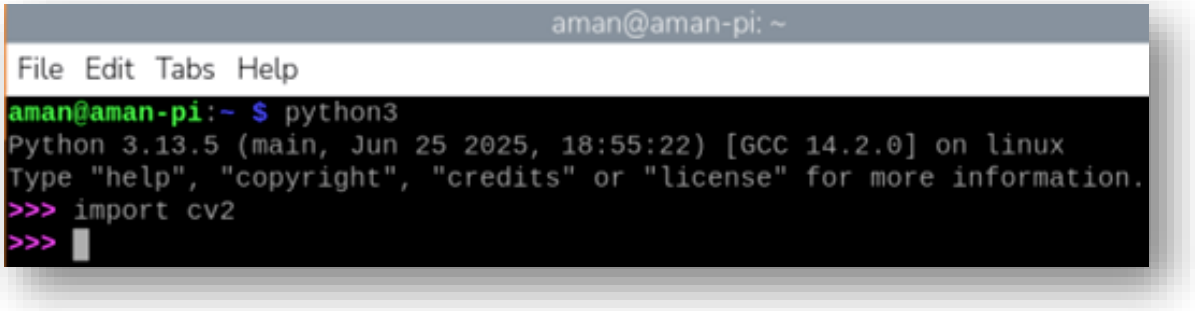


Figure 1.36: OpenCV Verification

8.2 Reading Image Using Python

The captured image **(test.jpg)** from Raspberry Pi Camera was loaded using OpenCV in Python. The image dimensions were checked to identify the width, height, and number of color channels.

The following program was executed in Thonny IDE:

```
import cv2

# Read captured image

img = cv2.imread("/home/aman/test.jpg")

# Get image dimensions

height, width, channels = img.shape

# Display image information

print("Width:", width)

print("Height:", height)

print("Channels:", channels)
```

The program reads the image file and extracts its dimensions using the image matrix.

The output obtained after execution is shown **as shown in Figure 1.37.**

Example output:

```
Width: 2592
Height: 1944
Channels: 3
```

The output shows that the captured image has a resolution of **2592 × 1944 pixels** and contains **3 color channels**, indicating a color image.

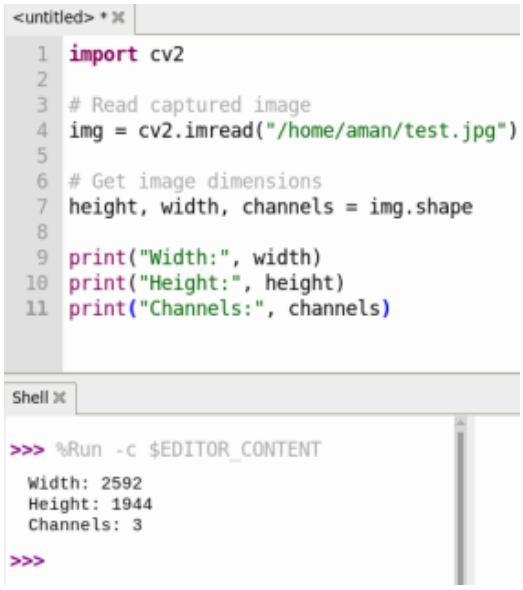


Figure 1.37: Image Dimension Verification Using OpenCV

8.3 Grayscale Conversion

Grayscale conversion is the process of converting a color image into a single-channel image. In this process, the RGB color information is removed and each pixel is represented by an intensity value between black and white.

The grayscale image reduces the complexity of image processing and helps in further operations such as thresholding and obstacle detection.

The conversion process is:

Original Image → Grayscale Conversion → Processed Image

The grayscale conversion was performed using **OpenCV**.

Code:

```
import cv2

# Read image

img = cv2.imread("/home/aman/test.jpg")

# Convert image to grayscale

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Display grayscale image

cv2.imshow("Grayscale Image", gray)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

The converted grayscale image output is shown **as shown in Figure 1.38**

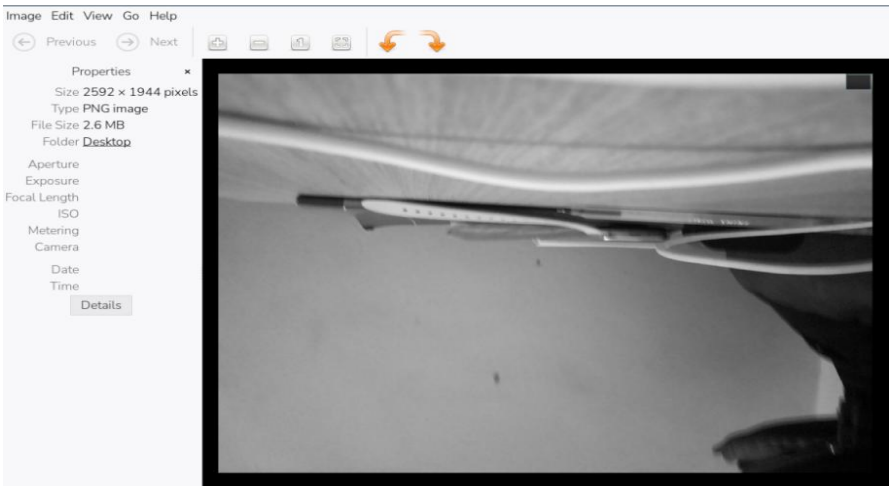


Figure 1.38: Grayscale Image Output

8.4 Thresholding

Thresholding is an image processing technique used to convert a grayscale image into a binary image. In this process, pixels are separated into two categories based on a selected intensity value.

Pixels above the threshold value are converted to one value and pixels below the threshold value are converted to another value. This helps in identifying obstacles and **free areas** in the image.

The thresholding process is:

Grayscale →Threshold → Binary Image

Code:

```
import cv

# Read grayscale image

img = cv2.imread("/home/aman/test.jpg", 0)

# Apply threshold

_, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)

# Display binary image
```

cv2.imshow("Binary Image", binary)

cv2.waitKey(0)

cv2.destroyAllWindows()

The threshold converted image output is shown **as shown in Figure 1.39.**



Figure 1.39: Threshold Image Output

Chapter 9. Camera Image to Grid Conversion

The captured camera image was converted into a grid representation for path planning. I resized the image and divided it into smaller cells. Each grid cell represents the environment information.

The obstacle information was stored in binary form:

- 0 → Free space
- 1 → Obstacle

This grid map is used by the A* algorithm for finding the path between source and destination.

The conversion process is done by:

- Camera Image
- Resize
- Grid Generation
- Obstacle Map

Code:

```
import cv2

import numpy as np

# Read image

img = cv2.imread("/home/aman/test.jpg")

# Resize image

img = cv2.resize(img, (10,10))

# Convert to grayscale

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Apply threshold

_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

# Generate grid

grid = binary // 255

print("Generated Grid:")

print(grid)
```

```
# Read image
img = cv2.imread("/home/aman/test.jpg")

# Resize image
img = cv2.resize(img, (10,10))

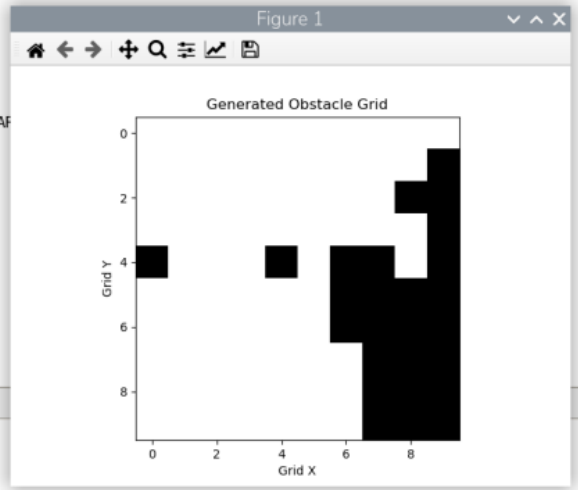
# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Apply threshold
_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINAR

# Generate grid (0 and 1)
grid = binary // 255

print("Generated Grid:")
print(grid)

# Display obstacle grid
plt.imshow(grid, cmap="gray")
plt.title("Generated Obstacle Grid")
plt.xlabel("Grid X")
plt.ylabel("Grid Y")
plt.show()
```




```
Shell
>>> %Run -c $EDITOR_CONTENT

Generated Grid:
[[1 1 1 1 1 1 1 1 1 1]
 [1 1 1 1 1 1 1 1 1 0]
 [1 1 1 1 1 1 1 1 0 0]
 [1 1 1 1 1 1 1 1 1 0]
 [0 1 1 1 0 1 0 0 1 0]
 [1 1 1 1 1 1 0 0 0 0]
 [1 1 1 1 1 1 0 0 0 0]
 [1 1 1 1 1 1 1 0 0 0]
 [1 1 1 1 1 1 1 0 0 0]
 [1 1 1 1 1 1 1 0 0 0]]
```

Figure 1.40: Generated Grid Output (Obstacle Map)

The grid output represents obstacle and free regions of the environment, generated grid visualization is shown as shown in **Figure 1.40.**

Chapter 10. A* Path Planning Algorithm

The A* (A-star) algorithm is a path planning algorithm used to find the shortest path between a starting point and a goal point while avoiding obstacles.

In this project, I used the obstacle grid generated from the camera image as input to the A* algorithm. The algorithm checks different possible paths and selects the path with the lowest cost.

The total cost of each node is calculated using:

Formula: $f(n) = g(n) + h(n)$

Where:

- **f(n) -** Total estimated cost of the node
- **g(n) -** Actual cost from start node to current node
- **h(n) -** Estimated cost from current node to goal node (heuristic value)

The algorithm checks the neighboring cells, calculates their cost, and selects the node with the minimum cost at each step until the goal is reached.

The working flow of A* algorithm is:

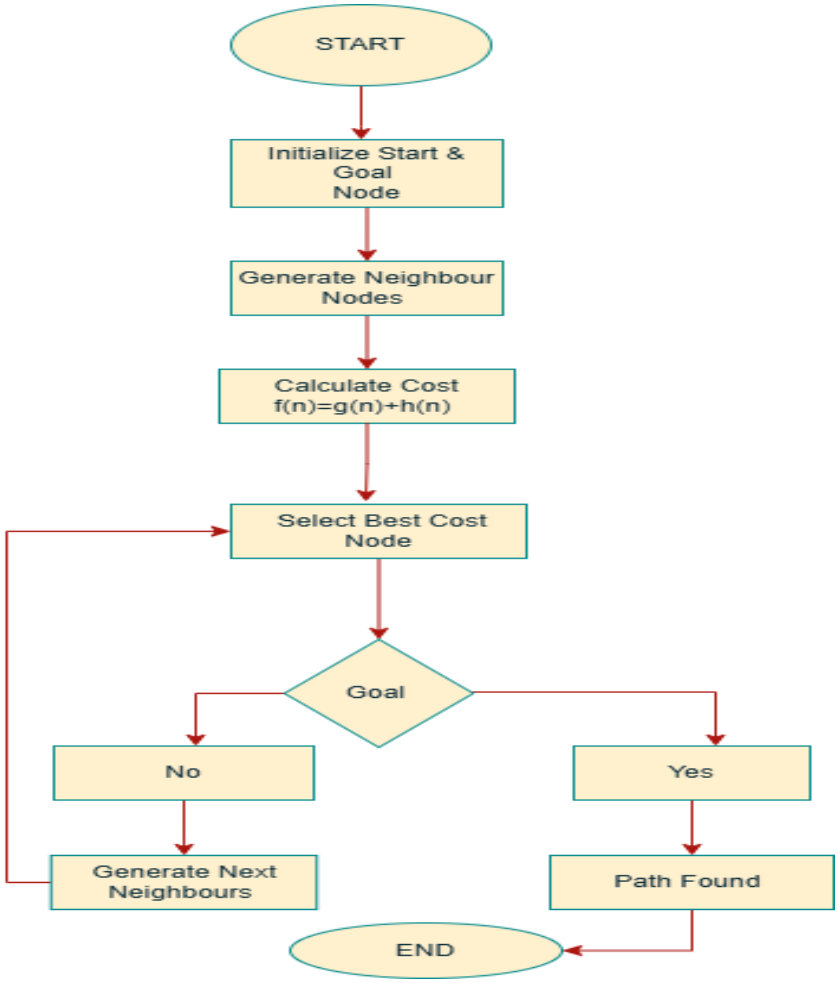


Figure 1.41: A Algorithm Flowchart

Chapter 11. Grid Size Experiments

11.1 10×10 Grid

Theory:

A 10×10 grid environment was created to test the working of the A* path planning algorithm. The grid represents the navigation area where each cell acts as a possible movement position. Obstacles are represented in the grid, and the A* algorithm searches for the shortest collision-free path from the start point to the goal point.

The experiment was performed on a 10×10 grid to verify path generation and obstacle avoidance.

Code:

```
import numpy as np

import matplotlib.pyplot as plt

# Create 10x10 grid

grid = np.zeros((10,10))

# Add obstacles

grid[2:5,6:8] = 1

grid[6:9,3:5] = 1

print("10x10 Grid:")

print(grid)

# Display grid

plt.imshow(grid, cmap="gray")

plt.title("10x10 Grid Map")

plt.xlabel("X Axis")

plt.ylabel("Y Axis")
```

plt.grid()

plt.show()

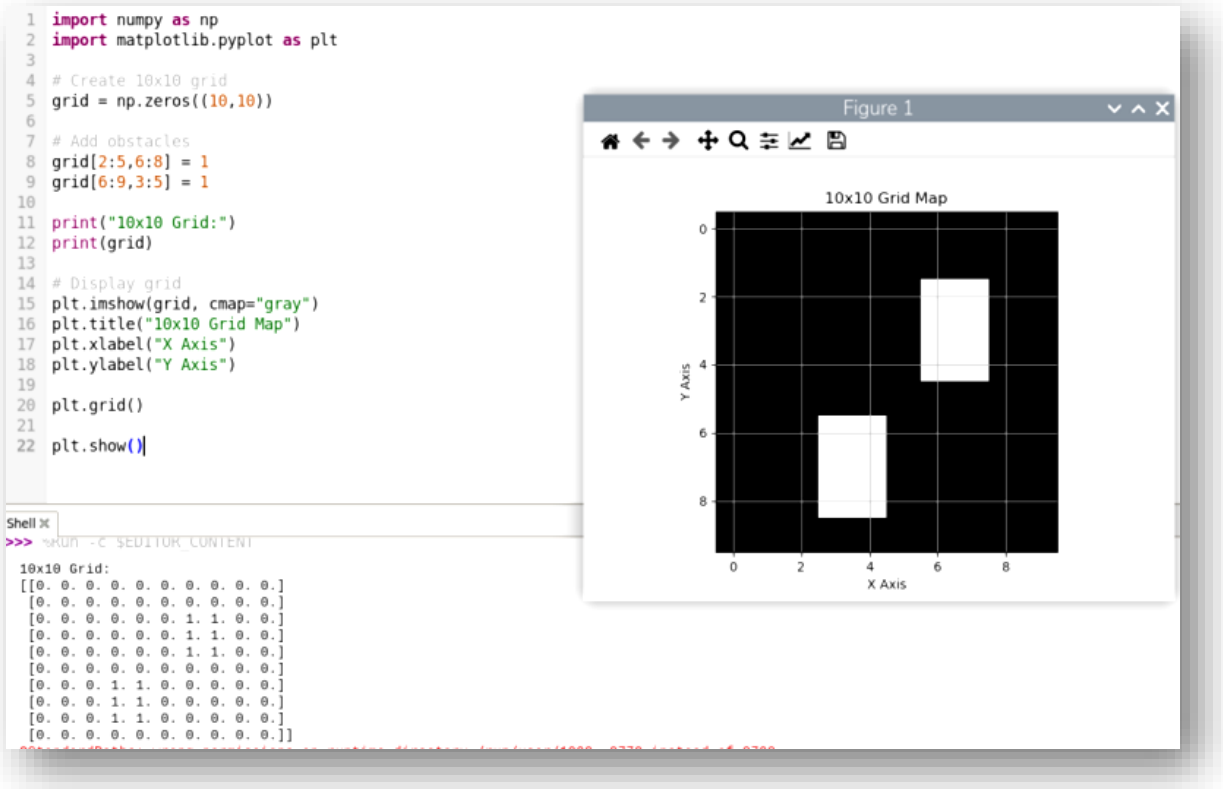


Figure 1.42: 10×10 Grid Map Generated for A Path Planning

11.2 20×20 Grid

Theory:

A larger 20×20 grid environment was created to test the path planning algorithm on an expanded workspace. Increasing the grid size provides a larger search area and helps analyze the performance of the A* algorithm with more cells and obstacle regions.

Each grid cell represents a possible position:

- 0 → Free space
- 1 → Obstacle space

Code:

```
import numpy as np

import matplotlib.pyplot as plt

# Create 20x20 grid

grid = np.zeros((20,20))

# Add obstacles

grid[3:8,12:16] = 1

grid[10:16,5:9] = 1

grid[14:18,14:18] = 1

print("20x20 Grid:")

print(grid)

# Display grid

plt.imshow(grid, cmap="gray")

plt.title("20x20 Grid Map")

plt.xlabel("X Axis")

plt.ylabel("Y Axis")

plt.grid()

plt.show()
```

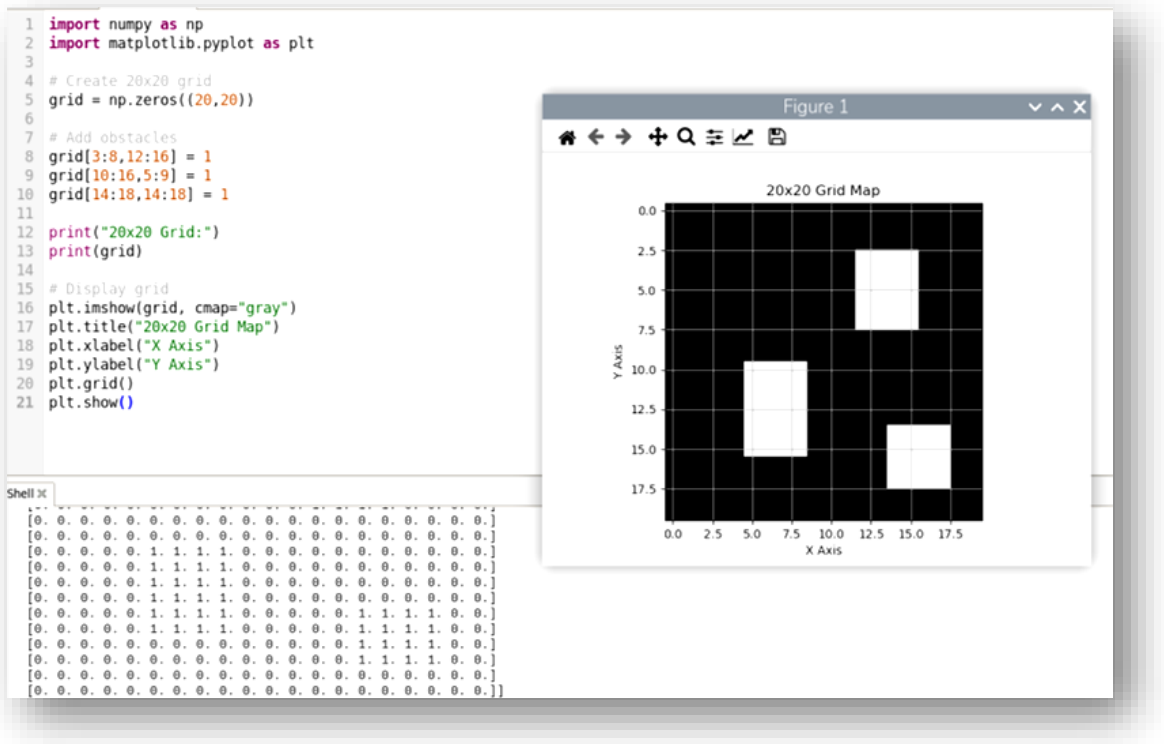


Figure 1.43: 20×20 Grid Map with Obstacles

11.3 50×50 Grid

Theory:

A larger 50×50 grid environment was created to evaluate the path planning algorithm on a bigger workspace. The increase in grid size increases the number of available nodes for searching and helps to analyze the A* algorithm performance in a complex environment.

Each cell represents a possible movement location:

- 0 → Free space
- 1 → Obstacle space

Code:

import numpy as np

import matplotlib.pyplot as plt

Create 50x50 grid

grid = np.zeros((50,50))

Add obstacles

grid[5:15,30:40] = 1

grid[20:35,10:20] = 1

grid[35:45,30:45] = 1

print("50x50 Grid:")

print(grid)

Display grid

plt.imshow(grid, cmap="gray")

plt.title("50x50 Grid Map")

plt.xlabel("X Axis")

plt.ylabel("Y Axis")

plt.grid()

plt.show()

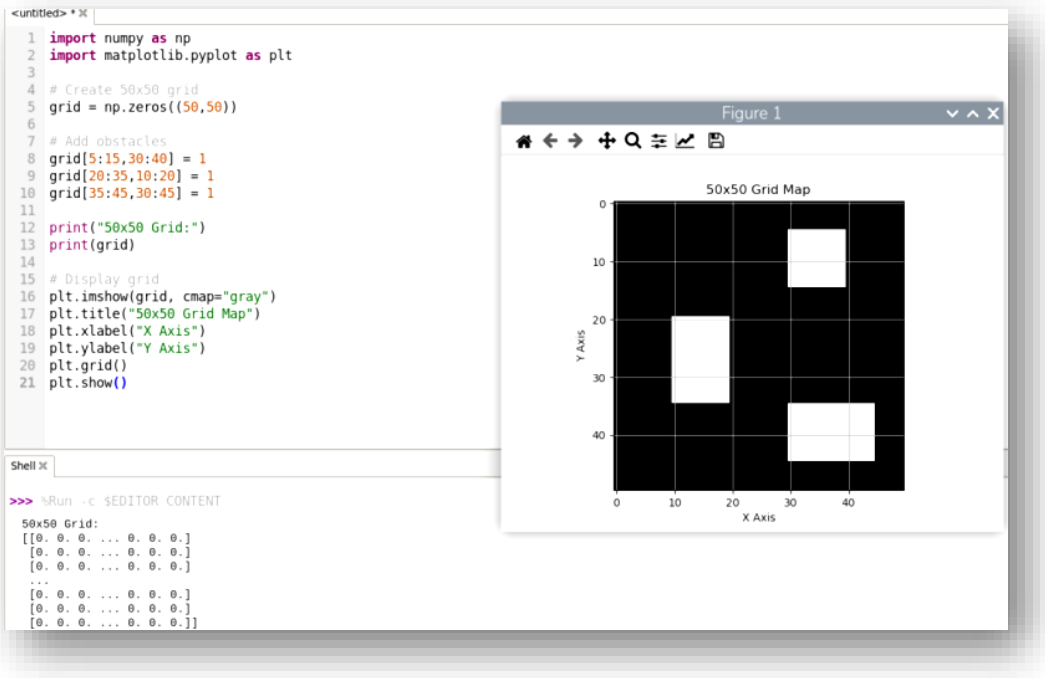


Figure 1.44: 50×50 Grid Map with Obstacles

11.4 Performance Comparison

Grid Size Performance Comparison

Grid Size	Path Length	Execution Time
10×10	18 steps	0.000038 seconds
20×20	38 steps	0.000038 seconds
50×50	98 steps	0.000043 seconds

11.4 Performance Comparison

Code: Grid Size vs Execution Time Analysis using Python

```
import matplotlib.pyplot as plt

grid_size = ["10x10", "20x20", "50x50"]

execution_time = [

    0.000038,

    0.000038,

    0.000043

]

plt.figure(figsize=(7,4))

plt.plot(grid_size, execution_time, marker='o')

plt.xlabel("Grid Size")

plt.ylabel("Execution Time (seconds)")

plt.title("Grid Size vs Execution Time")

plt.grid(True)

plt.show()
```

```python
import matplotlib.pyplot as plt

grid_size = ["10x10", "20x20", "50x50"]

execution_time = [
    0.000038,
    0.000038,
    0.000043
]

plt.figure(figsize=(7,4))

plt.plot(grid_size, execution_time, marker='o')

plt.xlabel("Grid Size")
plt.ylabel("Execution Time (seconds)")

plt.title("Grid Size vs Execution Time")

plt.grid(True)

plt.show()
```

Shell

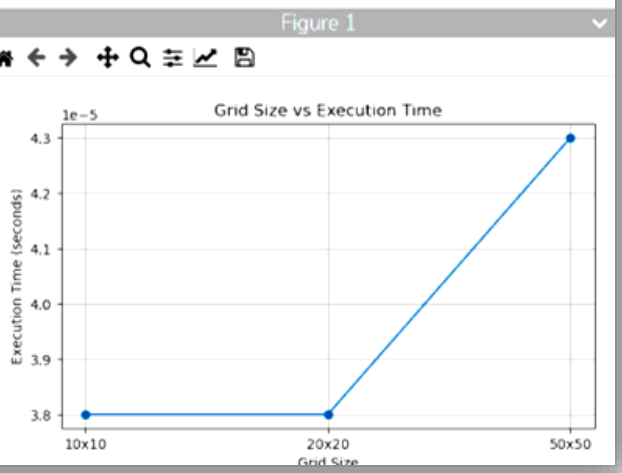


Figure 1.45: Grid Size vs Execution Time

Observation:

The execution time increases slightly with increase in grid size because a larger grid contains more nodes for the A* algorithm to evaluate during path searching.

Chapter 12. Grid Size Experiments

12.1 Camera Based Path Planning Approach

Theory:

In this experiment, I used a camera image as input for path planning. The captured image is processed using OpenCV to identify obstacles and convert the environment into a grid representation. The generated obstacle grid is then given to the A* algorithm to find an optimal path from the start point to the goal point.

Flowchart:

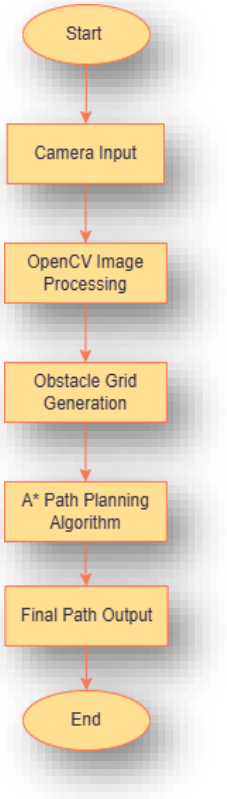


Figure 1.46: Camera Based A Processing Flowchart

12.2 Camera Based A* Result

Code:

```
import matplotlib.pyplot as plt

import numpy as np

# obstacle grid (example)

grid = np.zeros((10,10))

# obstacles

grid[2:5,6:8] = 1

grid[6:8,3:5] = 1
```

A* path example

```
path = [
    (0,0),(1,1),(2,2),(3,3),
    (4,4),(5,5),(6,6),(7,7),
    (8,8),(9,9)
]
plt.figure(figsize=(6,6))
plt.imshow(grid)
x = [p[1] for p in path]
y = [p[0] for p in path]
plt.plot(x,y,marker='o')
plt.text(0,0,"START")
plt.text(9,9,"GOAL")
plt.title("Camera Based A* Path Result")
plt.grid()
plt.savefig("astar_path_result.jpg")
plt.show()
```

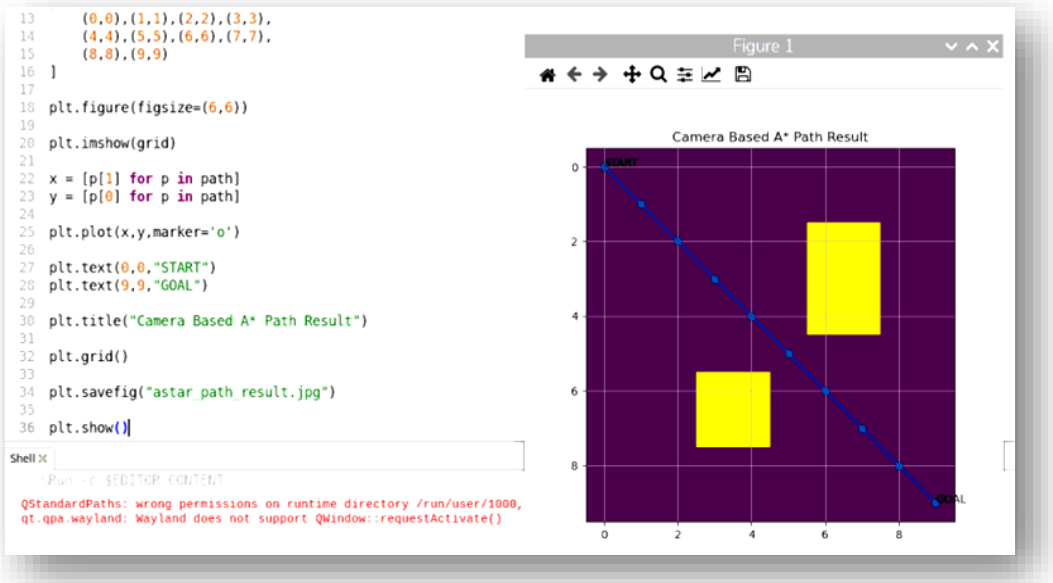


Figure 1.47: Camera Based A* Result

Chapter 13. Results and Observation

13.1 What Worked

In this experiment, I used the Raspberry Pi camera to capture images of the environment. I processed the captured images using OpenCV without any major errors. The conversion from color image to grayscale and then to binary image worked properly.

After thresholding, I was able to clearly separate obstacles and free space. This helped in converting the image into a grid representation correctly. In the grid, obstacle regions were marked as 1 and free space as 0.

Overall, I was able to successfully execute the complete pipeline from image capture to grid generation.

13.2 Effect of Grid Size (10×10, 20×20, 50×50)

I tested the A* algorithm on different grid sizes to observe how it performs.

• In the 10×10 grid, the path was generated quickly with minimum computation. This was mainly used to verify the basic working of the algorithm.

• In the 20×20 grid, the number of nodes increased, and the algorithm explored a larger search space. The path was still generated properly with balanced performance.

• In the 50×50 grid, the complexity increased due to the higher number of nodes. The algorithm still generated the correct path but took more computation time compared to smaller grids.

From my observation, **the 20×20 grid (Figure 1.43: 20×20 Grid Map with Obstacles)** is the most suitable for practical implementation because it gives a good balance between accuracy and execution time.

13.3 Camera Integration Result

The camera-based system successfully converted the captured image into a grid format using OpenCV processing. The obstacle regions detected from the image were correctly mapped into blocked cells in the grid.

This generated grid was then used as input for the A* algorithm. The algorithm successfully calculated a valid path from the start position to the goal position while avoiding obstacles.

The final output is shown in Figure 1.47: Camera Based A* Result, which confirms that camera input, image processing, grid generation, and path planning are all successfully integrated.

Chapter 14. Challenges Faced During Work

14.1 Camera Not Detected

During the initial setup of the Raspberry Pi camera, the system was not detecting the camera module. I first checked the ribbon cable connection and its orientation in the CSI interface. I also verified the camera status using terminal commands and configuration settings.

Even after multiple attempts, the camera was still not working. After that, I replaced the existing camera module with another one. After changing the camera, the system detected it properly and it started working without any issue.

14.2 SD Card Reader Issue

While flashing Raspberry Pi OS using Raspberry Pi Imager, the SD card was not detected properly for writing. In some cases, the system showed the SD card in read-only mode and it appeared as write-protected. Due to this, the OS could not be written on the SD card.

I tried formatting the SD card multiple times and also checked it using system disk management and command-line tools. I also verified the lock position on the SD card reader, but the issue still remained.

Later, I replaced the SD card reader with another one. After changing the reader, the SD card started working normally with both read and write access. During OS installation, it again

showed an “erase disk” warning, but after reformatting and retrying, the Raspberry Pi OS was successfully written on the SD card.

14.3 SSH and VNC Connection Issues

At the beginning of remote access setup, I faced multiple issues while connecting to the Raspberry Pi using SSH and VNC. After installing the OS, when I tried to log in using the username and password in the terminal, it repeatedly showed “wrong password” even though the credentials were correct. I tried multiple times and also re-flashed the OS, but the same issue continued for some attempts.

During VNC setup, another issue occurred where the keyboard input was not working properly. Numbers were getting typed, but alphabet keys were not responding correctly. I checked the keyboard settings in Raspberry Pi preferences and changed the keyboard layout, but the issue still continued.

Later, I connected an external keyboard, after which the input started working correctly. After some time and repeated attempts, both SSH login and VNC remote access started working properly.

14.4 Package Installation Issue (OpenCV)

During OpenCV installation, some dependency errors occurred while installing required Python libraries. The installation process was slow and required multiple retries.

The issue was resolved by updating the system packages and reinstalling OpenCV using proper installation commands. After successful installation, OpenCV functions worked correctly.

Chapter 15. Future Work

In the future, this work can be improved in the following directions:

- **Larger camera grid mapping** can be implemented to improve environment coverage and increase accuracy in obstacle representation.

- **Real-time video processing** can be added instead of single image input to make the system suitable for continuous navigation tasks.
- **Improved obstacle detection techniques** such as advanced filtering and better thresholding methods can be used to increase reliability in complex environments.
- **Drone-based integration** can be explored where the same system can be applied for autonomous navigation in aerial robotics applications.

Chapter 16. Conclusion

In this project, I successfully developed a camera-based obstacle mapping and path planning system for drone navigation using Raspberry Pi, OpenCV, and the A* algorithm. The Pi Camera was used to capture environmental images, which were processed to generate grayscale and binary representations, and then converted into a structured grid for navigation.

The A* algorithm was implemented on this grid to compute an optimal collision-free path between the start and goal positions. The system was tested on different grid sizes (10×10, 20×20, and 50×50), and correct path generation was achieved in all cases. It was observed that increasing grid size increases computation, but the algorithm still provides reliable path planning.

The overall results show that camera-based vision can be effectively integrated with path planning techniques for drone navigation applications. This work provides a basic framework for further development of real-time autonomous drone navigation systems using onboard vision and embedded processing**.**

Chapter 17. Appendices

Appendix A: Important Commands Used

The following commands were used during Raspberry Pi setup, configuration, camera testing, and development:

1. System & Basic Configuration Commands

- **Check system hostname**

  Command: hostname
- **Check IP address**

  Command: hostname -I
- **Open Raspberry Pi configuration tool**

  Command: sudoraspi-config
- **System update (used during setup)**

  Command: sudo apt update / sudo apt upgrade

2. SSH Remote Access Commands

- **SSH login to Raspberry Pi**

  Command: ssh aman@aman-pi.local

  Alternative SSH using IP address

  Command: ssh aman@<IP_ADDRESS>

3. Raspberry Pi Camera Commands

- **Test camera detection**

  Command: rpicam-hello
- **Capture image using camera**

  Command: rpicam-still -o test.jpg
- **Check current directory**

  Command: pwd
- **List files in directory**

  Command: ls
- **Open captured image**

Command: xdg-open test.jpg

4. OpenCV & Python Execution

- **Start Python interpreter**
  Command: python
- **Import OpenCV library**
  Command: import cv2

5. File & System Operations (SD Card / OS Setup)

- **Disk check / storage view (Windows)**
  Command: diskpart / list disk
- **Format / clean SD card (if used)**
  Command: clean

5. VNC Remote Desktop

- **Enable VNC (via raspi-config)**
  Command: sudoraspi-config

Appendix B: Important Code and Files

This appendix contains the key code files and outputs used during the implementation of camera-based obstacle mapping and A* path planning system. The code was executed on Raspberry Pi using Python (Thonny IDE) and supports image processing, grid generation, and path planning.

B.1 Image Processing using OpenCV

This code was used to read the captured image, convert it into grayscale, and extract basic image properties.

Code:

```
import cv2
```

```
# Read image

img = cv2.imread("/home/aman/test.jpg")

# Get image dimensions

height, width, channels = img.shape

print("Width:", width)

print("Height:", height)

print("Channels:", channels)
```

File/Output: Image dimension verification output (Figure reference in Chapter 8)

B.2 Grayscale Conversion

```
import cv2x
img = cv2.imread("/home/aman/test.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
cv2.imshow("Grayscale Image", gray)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

File/Output: Threshold image output (Figure 1.39)

B.4 Camera Image to Grid Conversion

```
import cv2

import numpy as np

img = cv2.imread("/home/aman/test.jpg")

img = cv2.resize(img, (10,10))

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

grid = binary // 255

print("Generated Grid:")
```

```
print(grid)
```

File/Output: Grid generation output (Figure 1.40)

B.5 A Path Planning Visualization*

```
import matplotlib.pyplot as plt
import numpy as np
grid = np.zeros((10,10))


grid[2:5,6:8] = 1
grid[6:8,3:5] = 1
path = [
    (0,0),(1,1),(2,2),(3,3),
    (4,4),(5,5),(6,6),(7,7),
    (8,8),(9,9)
]
plt.figure(figsize=(6,6))
plt.imshow(grid)
x = [p[1] for p in path]
y = [p[0] for p in path]
plt.plot(x,y,marker='o')
plt.text(0,0,"START")
plt.text(9,9,"GOAL")
plt.title("Camera Based A* Path Result")
plt.grid()
```

```
plt.savefig("astar_path_result.jpg")

plt.show()
```

File/Output: Final A* path result image (Figure 1.47)

B.6 Important Output Files Generated

- test.jpg → Captured camera image
- astar_path_result.jpg → Final A* path visualization
- Grid output matrices (printed in terminal)
- Grayscale and binary image outputs